

PISA: PORTABLE IoT SECURITY ASSESSMENT PLATFORM

*An End-to-End System for Wireless Network Posture Scoring, IoT Device
Fingerprinting, and Authorized Vulnerability Verification*

A PROJECT REPORT

Submitted by

VIVEK REDDY

(CH.SC.U4CYS23034)

in partial fulfillment for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)**

Supervisor

Dr. Jinka Aravind

Submitted to

**AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
CHENNAI – 601103**

November 2026

AMRITA SCHOOL OF COMPUTING, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report "PISA: PORTABLE IoT SECURITY ASSESSMENT PLATFORM" is the bonafide work of "VIVEK REDDY (CH.SC.U4CYS23034)" who carried out the project work under my supervision.

<<Signature of Head of the Department>>

Dr. S. Udhaya Kumar

Program Chair, Department of Computer
Science and Engineering (Cyber Security)
Amrita School of Computing, Amrita Vishwa
Vidyapeetham, Chennai – 601103

<<Signature of Supervisor>>

Dr. Jinka Aravind

Assistant Professor, Department of Computer
Science and Engineering
Amrita School of Computing, Amrita Vishwa
Vidyapeetham, Chennai – 601103

INTERNAL EXAMINER

EXTERNAL EXAMINER

AMRITA SCHOOL OF COMPUTING, CHENNAI

DECLARATION BY THE CANDIDATE

I declare that the report entitled "PISA: PORTABLE IoT SECURITY ASSESSMENT PLATFORM" submitted by me for the degree of Bachelor of Technology in Computer Science and Engineering (Cyber Security) is the record of the project work carried out by me under the guidance of "Dr. Jinka Aravind" (Assistant Professor, Department of Computer Science and Engineering), and that this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or similar title in this or any other University or other similar institution of higher learning.

<<Signature of Student>>

VIVEK REDDY

Reg. No. CH.SC.U4CYS23034

ABSTRACT

The proliferation of Internet of Things (IoT) devices has created an attack surface with two layers that existing security tooling treats in complete isolation: the wireless network a device connects through, and the device itself. No published tool scores a network's security posture from passive observation alone, identifies the IoT devices on it without a machine-learning model, correlates those devices against real, currently-exploited vulnerabilities, and safely verifies exploitability — all within a single pipeline on portable, field-deployable hardware. This report presents PISA (Portable IoT Security Assessment Platform), a system built to close that gap on Raspberry Pi 4 class hardware. PISA is organized as six modules: M0 computes a WiFi Security Posture Score (WSPS, A–F) from passive 802.11 beacon analysis and correlates an access point's vendor against known CVEs; M1 joins a scored network and discovers its devices via ARP sweep, Nmap, and mDNS; M2 fingerprints each device's MQTT, CoAP, HTTP, and RTSP behaviour and fuses per-protocol confidence using a probabilistic complement rule; M3 correlates devices against the live National Vulnerability Database, enriches results with FIRST.org's Exploit Prediction Scoring System (EPSS) and the CISA Known Exploited Vulnerabilities catalogue, and evaluates real NVD configuration logic before ever reporting a device as affected; and M4 verifies and, only under explicit, logged operator authorization, exploits a confirmed vulnerability through RouterSploit, gated by a named-operator confirmation and a mandatory countdown. All six modules are implemented, covered by 381 automated tests, and have each been validated against real hardware, real networks, and real external APIs: four real campus networks scored by WSPS, a real IP camera fingerprinted at full confidence, a real reference CVE (CVSS 9.8, EPSS 29%) correctly matched and an honest non-match correctly refused, and the authorization gate shown, through adversarial live testing, to block every unauthorized or under-evidenced exploitation attempt tried against it. The report positions PISA against twenty peer-reviewed sources and nine named prior tools, states five specific novelty claims, and reports results and limitations honestly as a basis for the work remaining before the final project review.

Keywords: IoT Security, WiFi Security Posture Scoring, Protocol Behavioural Fingerprinting, Vulnerability Correlation, EPSS, CISA KEV, Authorized Exploitation, Penetration Testing, Edge Security, Raspberry Pi.

ACKNOWLEDGEMENT

This project would not have been possible without the contribution of many people. It gives me immense pleasure to express my profound gratitude to our honourable Chancellor, **Sri Mata Amritanandamayi Devi**, for her blessings and for being a source of inspiration. I extend my sincere thanks to Administrative Director, **Sampoojya Swami Vinyamritananda Puri**, of Amrita Vishwa Vidyapeetham, Chennai, for his guidance and support. I am deeply indebted to our Director, **I B Manikantan**, of Amrita School of Computing, for providing all the facilities and extended support that enabled me to gain valuable education and learning experience.

I extend my special thanks to **Dr. V Jayakumar**, Principal of Amrita School of Computing, for the support provided in the successful completion of this project. I am also grateful to **Dr. S Baghavathi Priya**, Chairperson, Department of Computer Science and Engineering, and **Dr. S. Udhaya Kumar**, Program Chair, Department of Computer Science and Engineering (Cyber Security), for their continued encouragement throughout this work.

I am sincerely thankful to my supervisor, **Dr. Jinka Aravind**, Assistant Professor, Department of Computer Science and Engineering, for his valuable guidance, constructive feedback, and constant support at every stage of this project, from the initial problem scoping through to the real-hardware validation reported in this report. Finally, I thank my family and friends for their encouragement and support throughout this project.

VIVEK REDDY

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	INTRODUCTION	1
	BACKGROUND AND MOTIVATION	1
	PROBLEM STATEMENT	2
	OBJECTIVES OF THE PROJECT	3
	SCOPE OF THE PROJECT	4
	In Scope for the Current Phase	4
	Out of Scope for the Current Phase	5
	PROJECT CONSTRAINTS	5
	SIGNIFICANCE OF THE STUDY	6
	ORGANIZATION OF THE REPORT	6
2	LITERATURE REVIEW	8
	INTRODUCTION	8
	WIFI SECURITY ASSESSMENT AND BEACON-LEVEL ANALYSIS	8
	NETWORK DISCOVERY AND THE ENTERPRISE IOT ATTACK SURFACE	9
	PROTOCOL-AWARE IOT DEVICE FINGERPRINTING	10
	Traffic-Flow and Machine-Learning Approaches	10
	Deterministic and Protocol-Specific Fingerprinting	10
	CVE CORRELATION AND EXPLOIT SCORING	11
	EXPLOIT VALIDATION AND AUTHORIZATION	12
	UNIFIED IOT SECURITY AUDIT TOOLING	12
	COMPARATIVE ANALYSIS OF EXISTING TOOLS AND PLATFORMS	12
	CONSOLIDATED VIEW OF THE REVIEWED LITERATURE	14
	SUMMARY OF RESEARCH GAPS	15
3	PROBLEM STATEMENT AND METHODOLOGY	17
	PROBLEM DEFINITION	17
	RESEARCH QUESTIONS AND HYPOTHESES	17
	DEVELOPMENT METHODOLOGY	18
	NOVELTY CLAIMS	20
	Claim 1 — WiFi Security Posture Scoring (WSPS)	20
	Claim 2 — OUI-to-CVE Infrastructure Correlation	20
	Claim 3 — Non-ML Cross-Protocol Behavioural Fingerprinting	21
	Claim 4 — Integrated Two-Layer Pipeline on Portable Hardware	21
	Claim 5 — Tri-Metric Exploitability Scoring at the Edge	21
	REQUIREMENTS SPECIFICATION	21
	ETHICS, LEGAL CONSTRAINTS, AND SAFETY	22
4	SYSTEM DESIGN AND IMPLEMENTATION	24

	SYSTEM ARCHITECTURE OVERVIEW	24
	HARDWARE ARCHITECTURE	25
	SOFTWARE ARCHITECTURE AND TECHNOLOGY STACK	26
	MODULE DESIGN	27
	M0 — WiFi Assessment Layer	27
	M1 — Network Discovery	29
	M2 — Protocol-Aware Behavioural Fingerprinting	30
	M3 — Vulnerability Intelligence	31
	M4 — Authorized Verification and Gated Exploitation	32
	M5 — Dashboard and Reporting	33
	DATABASE DESIGN	34
	SECURITY AND AUTHORIZATION GATE DESIGN	35
	TESTING STRATEGY AND CONTINUOUS INTEGRATION	35
	DEPLOYMENT AND OPERATION	36
	SUMMARY OF ENGINEERING DESIGN DECISIONS	37
5	RESULTS AND DISCUSSION	38
	IMPLEMENTATION STATUS SUMMARY	38
	REAL HARDWARE VALIDATION RESULTS	38
	M0 — WiFi Assessment	38
	M1 — Network Discovery	39
	M2 — Protocol Fingerprinting	39
	M3 — Vulnerability Intelligence	39
	M4 — Verification and Gated Exploitation	39
	M5 — Dashboard / API	40
	AUTOMATED TEST SUITE SUMMARY	40
	SECURITY GATE — ADVERSARIAL TESTING RESULTS	41
	DISCUSSION	42
	KNOWN LIMITATIONS	43
	ILLUSTRATIVE END-TO-END SESSION WALKTHROUGH	44
	THREATS TO VALIDITY	44
6	CONCLUSION AND SCOPE FOR FUTURE WORK	46
	CONCLUSION	46
	SUMMARY OF CONTRIBUTIONS	47
	ROADMAP TO THE FINAL REVIEW	47
	FUTURE WORK BEYOND THE FINAL REVIEW	47
	REFERENCES	49
	APPENDICES	52
	PLAGIARISM REPORT	52
	REPOSITORY AND DOCUMENTATION INDEX	52

LIST OF TABLES

Table No.	Title	Page No.
1.1	Capability gaps addressed by this project	3
1.2	Project constraints and how PISA satisfies them	5
2.1	PISA compared against existing tools and platforms	13
2.2	Consolidated summary of the twenty reviewed sources	14
3.1	Planned evaluation metrics per hypothesis	18
3.2	Software Requirements Specification revision history	19
3.3	Technical risks encountered and their mitigations	20
3.4	Functional requirements summary (v1)	21
3.5	Non-functional requirements summary (v1)	22
4.1	Hardware specification	25
4.2	Indicative hardware cost (components not already owned)	26
4.3	Core software dependencies (requirements.txt)	26
4.4	WSPS scoring factors (pisa/m0/wsps.py)	27
4.5	M1-to-M2 port-to-protocol routing rule	29
4.6	Representative behavioural signatures probed per protocol	31
4.7	SQLite schema (pisa/db/models.py)	34
4.8	Key engineering design decisions and their rationale	37
5.1	Module implementation status at the current review	38
5.2	Automated test count by module (live pytest --collect-only)	40
5.3	Adversarial gate testing results (live HTTP against the running dashboard)	41
5.4	Hypothesis validation status at the current review	42
6.1	Roadmap to the final review	47

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	PISA five-layer system architecture	24
4.2	M0 (WiSentinel) internal processing pipeline	28
4.3	Per-module processing pipelines, M0 through M5	30
4.4	M5 interface and reporting engine	34

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Abbreviation	Expansion
IoT	Internet of Things
WSPS	WiFi Security Posture Score
OUI	Organizationally Unique Identifier
BSSID	Basic Service Set Identifier
SSID	Service Set Identifier
PMF	Protected Management Frames (IEEE 802.11w)
WPS	WiFi Protected Setup
RSSI	Received Signal Strength Indicator
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CPE	Common Platform Enumeration
NVD	National Vulnerability Database
EPSS	Exploit Prediction Scoring System
KEV	Known Exploited Vulnerabilities (CISA catalogue)
MQTT	Message Queuing Telemetry Transport
CoAP	Constrained Application Protocol
RTSP	Real Time Streaming Protocol
HTTP	Hypertext Transfer Protocol
ARP	Address Resolution Protocol
mDNS	Multicast Domain Name System
API	Application Programming Interface
SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement
CI/CD	Continuous Integration / Continuous Deployment
AWS	Amazon Web Services

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

The number of Internet of Things (IoT) devices deployed across homes, enterprises, and industrial environments has grown far faster than the security practices needed to manage them. Smart plugs, IP cameras, home routers, and sensor hubs are routinely shipped with default credentials, outdated firmware, and application-layer protocols such as MQTT and CoAP that were never designed with an adversarial network in mind. The United States Cybersecurity and Infrastructure Security Agency (CISA) maintains a Known Exploited Vulnerabilities (KEV) catalogue that, as of this project's survey of the literature, lists hundreds of router and IoT-device CVEs confirmed to be under active exploitation in the wild, yet no widely available tool allows a field assessor to determine, in minutes and on portable hardware, whether a specific environment is exposed to any of them.

This gap is not for lack of individual tools. Wardriving utilities such as Kismet and WiGLE log raw WiFi beacon data without any security scoring; network scanners such as Nmap enumerate open ports and services without correlating them to known vulnerabilities; vulnerability databases such as the National Vulnerability Database (NVD) and exploitation frameworks such as RouterSploit exist in complete isolation from one another. An assessor wishing to answer the simple question "is this network, and the devices on it, safe" today has to manually stitch together four or more separate tools, each with its own mental model of what a "target" is, and no shared chain of evidence connecting a discovered weakness to a verified, exploitable finding. This fragmentation is itself a documented research gap: a 2024 systematic review of more than forty IoT security audit tools concluded that no existing tool integrates wireless-network-layer assessment with device-layer vulnerability correlation in a single, portable pipeline [20].

PISA (Portable IoT Security Assessment Platform) is the system developed in response to this gap. It is designed to run on commodity, field-deployable hardware (a Raspberry Pi 4 together with an external monitor-mode WiFi adapter) and to carry an assessor through a single, evidence-preserving pipeline: passively score the security posture of a wireless network from its beacon frames alone, correlate the access point's hardware vendor against known CVEs, join the network on request to enumerate and fingerprint the IoT devices living on it, correlate those devices' CVEs using NVD, FIRST.org's Exploit Prediction Scoring

System (EPSS) and the CISA KEV catalogue, and — only under explicit, logged operator authorization — verify specific vulnerabilities against those devices before any exploitation is even offered. Every stage of this pipeline is architecturally in the spirit of standalone penetration-testing hardware (in the manner of Hak5-style gadgets) rather than a laptop-bound application, and the whole assessment is presented through a single browser dashboard.

1.2 PROBLEM STATEMENT

The IoT attack surface has two layers that existing tooling treats in complete isolation. The first is the **network access layer**: the WiFi network an IoT device connects to is itself frequently misconfigured, running deprecated encryption, leaving Wi-Fi Protected Setup (WPS) enabled, or running access-point firmware with publicly known, actively exploited CVEs. Even WPA3, the current state of the art, has been shown to carry exploitable side-channel and downgrade weaknesses in its Dragonfly handshake [19], and systematic analysis of pre-authentication WiFi behaviour has confirmed that management frames such as beacons leak substantial security-relevant information before a device ever associates with a network [5]. No portable tool autonomously assesses this layer and expresses the result as a single, actionable security grade.

The second is the **device layer**: once inside a network, IoT devices communicate using application-layer protocols — MQTT, CoAP, HTTP, RTSP — that carry behavioural signatures specific to their manufacturer and firmware generation. Existing approaches to identifying these devices either depend on machine learning and large labelled traffic corpora [15], or stop at confirming which service ports are open without analysing how a device behaves within its own protocol. Compounding this, large-scale measurement of MQTT broker deployments has found that the large majority run with no authentication configured at all [11], and CoAP's plaintext, connectionless design exposes a comparable resource-discovery surface [1]. Translating "this device is a Hikvision camera" into "this device is affected by CVE-2021-36260" is itself non-trivial: a device fingerprint has to be mapped to a precise Common Platform Enumeration (CPE) string, correlated against NVD, and ranked by real-world exploitability rather than severity alone, since EPSS scores are frequently low or unavailable for CVEs at the moment they first appear in the CISA KEV catalogue [17] — meaning no single scoring metric is sufficient on its own.

No existing tool, product, or published work performs an end-to-end, automated assessment spanning both layers — from passive wireless posture scoring through device fingerprinting, precise CVE correlation, and safe exploit verification — on portable, field-deployable hardware. This leaves security practitioners, and academic researchers studying IoT security at scale, without an efficient methodology for comprehensive environment auditing that does not depend on a laptop, an internet-scale cloud service, or a trained machine-learning model. Table 1.1 summarizes the specific capability gaps this project addresses and is examined in full, against named prior tools and published work, in Chapter 2.

Table 1.1: Capability gaps addressed by this project

Gap	Existing practice	Consequence
No unified WiFi + device pipeline	Nmap, Bettercap, RouterSploit, manual NVD search used as four disconnected tools	No shared evidence model from discovery to exploitation
No passive WiFi posture scoring	Wardriving tools (Kismet, WiGLE) log raw beacon data only	No actionable A-F grade an assessor can report
No non-ML cross-protocol fingerprinting	ML-based classifiers require labelled training data per device class	Cannot identify unseen or custom industrial/IoT devices
No contextual exploitability ranking	CVSS severity alone is used to prioritize findings	Low-severity, actively-exploited CVEs (per CISA KEV) are missed
No gated, auditable exploitation step	Manual exploit selection with no enforced authorization state machine	No defensible, logged chain of authorization before any live exploit attempt

Each row in Table 1.1 is a gap in current practice, not a gap in any single tool's feature list — meaning it cannot be closed by adding a feature to Nmap or to RouterSploit individually, since the missing element is the connective evidence chain between them, not a missing capability within either. This framing matters for how PISA's contribution should be read throughout this report: Chapter 4 does not claim to out-perform Nmap at scanning or RouterSploit at exploitation — both are reused directly as execution backends, as detailed in Section 4.3 — but to be the first system that connects what they each already do into one pipeline that never loses track of which stage actually established which piece of evidence.

1.3 OBJECTIVES OF THE PROJECT

The primary objective of this project is to design, implement, and evaluate a portable, self-contained IoT security assessment platform on Raspberry Pi 4 class hardware that performs automated, two-layer security assessment: wireless-network vulnerability scoring

and access-point CVE correlation on one hand, and IoT device protocol-aware behavioural fingerprinting with CVE correlation and gated exploit verification on the other.

This primary objective is decomposed into the following secondary objectives.

- **O1.** Develop a WiFi Security Posture Scoring (WSPS) framework that derives a quantitative, letter-graded (A-F) security score for a network from passive beacon frame analysis alone, without active association, authentication, or packet injection.
- **O2.** Implement protocol-aware behavioural fingerprinting for IoT devices across MQTT, CoAP, HTTP, and RTSP that identifies device type and, where evidence supports it, manufacturer and model, without using machine learning or a labelled training corpus.
- **O3.** Build an automated device-fingerprint-to-CVE correlation pipeline against the live NVD CVE API, enriched with FIRST.org's EPSS exploitation-probability score and the CISA KEV catalogue, combined into a single contextual exploitability metric.
- **O4.** Implement a safe, explicitly authorized exploit-verification stage that structurally prevents a device from being reported as exploitable unless it has first been found applicable by the CVE engine and then live-verified, with every authorization and outcome recorded in an append-only audit log.
- **O5.** Validate the complete pipeline against real hardware, real live networks, and real external threat-intelligence APIs (NVD, FIRST.org EPSS, CISA KEV) rather than synthetic or simulated data, and report results exactly as observed.

1.4 SCOPE OF THE PROJECT

1.4.1 In Scope for the Current Phase

The current release (v1) of PISA, which this report documents, implements six software modules, referred to throughout this report as M0 through M5, described in full in Chapter 4. In summary, the system in its present form: captures 802.11 beacon frames passively and computes a WSPS grade per network (M0); resolves an access point's vendor from its BSSID and performs an on-demand NVD CVE lookup against that vendor (M0); joins a scored network with an operator-supplied password and discovers live devices on it using an ARP sweep and an Nmap service/OS scan, identified further via mDNS (M1); fingerprints each discovered device's MQTT, CoAP, HTTP, and RTSP behaviour and fuses the per-protocol evidence into a single device-type and confidence (M2); correlates each device against NVD, enriches every returned CVE with its EPSS score and CISA KEV status, and computes a combined exploit score (M3); and, only under an operator's explicit, logged authorization,

verifies specific CVEs against a device through RouterSploit and performs a gated exploitation attempt (M4). All of the above is presented and triggered through a single local Flask dashboard (M5).

1.4.2 Out of Scope for the Current Phase

Cloud-backed reporting and long-term storage (synchronization of session data to AWS DynamoDB/S3 and automated PDF report generation via AWS Lambda) is architecturally scoped but intentionally not implemented in v1; the ``pisa/aws/`` package exists as a stub only, and is treated as future work in Chapter 6. Likewise, GPS-tagged "warwalking", a dedicated touchscreen interface, continuous background monitoring mode, and any machine-learning- or blockchain-based component are explicitly out of scope — the last two by a project-level constraint to keep every classification and scoring decision deterministic and auditable, which is itself revisited as a design principle in Chapter 3.

1.5 PROJECT CONSTRAINTS

This project was undertaken under a fixed set of technology and domain constraints set for the Bachelor of Technology final-year project track it belongs to. Table 1.2 states each constraint and how PISA's design, described fully in Chapter 4, satisfies it; these constraints directly shaped several of the scope decisions explained in Section 1.4 — most visibly, the decision to keep every classification and scoring decision in Chapters 4 and 5 deterministic and rule-traceable rather than learned.

Table 1.2: Project constraints and how PISA satisfies them

Constraint	How PISA satisfies it
Cloud platform (AWS or Azure)	AWS integration (DynamoDB, S3, Lambda) is architected in Chapter 4 and scoped as the project's next major extension in Chapter 6; boto3 is already a pinned dependency
CI/CD and DevOps practice	GitHub Actions runs the full automated test suite on every push/PR (Section 4.7)
Raspberry Pi 4 and peripheral hardware	RPi 4 plus an external monitor-mode WiFi adapter is the system's core target platform (Section 4.2)
IoT security / hardware security domain	The project's entire scope, from Chapter 1 onward
No machine learning or deep learning as a core mechanism	Every classification and scoring decision (WSPS, device fingerprinting, CVE applicability) is additive, rule-based, and individually traceable (Chapter 4)
No blockchain component	Not used anywhere in the system
No dataset-based offline analysis	All assessment is performed against live, active capture and live external APIs, never a static/offline dataset (Chapter 5)

1.6 SIGNIFICANCE OF THE STUDY

This project is significant along two distinct axes. Practically, it addresses a concrete, named shortfall in current security-assessment practice: the manual, multi-tool workflow described in Section 1.2 is not merely inconvenient, it structurally loses the evidence chain between discovery and exploitation, meaning an assessor using Nmap, Bettercap, and RouterSploit separately has no systematic guarantee against conflating "this CVE's keyword matched this device's banner" with "this device is actually vulnerable" — a distinction Chapter 4 shows PISA enforces structurally rather than by operator discipline alone. A field assessor, a small organization without a dedicated security team, or a student studying IoT security stands to benefit directly from a single, portable tool that preserves this distinction automatically.

Academically, the project contributes a specific, falsifiable set of claims — the five novelty claims formalized in Chapter 3 — each positioned against named prior work rather than asserted in isolation, and each paired with a result reported honestly in Chapter 5 as validated, partially validated, or not yet validated. This discipline is itself part of the contribution: a project report that only ever reports successes is weaker evidence of engineering competence than one that states, in the same document, what has been proven and what remains open, which is the standard this report holds itself to throughout.

1.7 ORGANIZATION OF THE REPORT

The remainder of this report is organized as follows. Chapter 2 surveys the published literature underpinning each layer of PISA's design and positions the project against named prior tools and research systems. Chapter 3 formalizes the problem statement into concrete research questions and hypotheses, states the project's novelty claims, and summarizes the functional and non-functional requirements and the ethical/legal constraints the system is designed under. Chapter 4 describes the system architecture, hardware, technology stack, and the design of each of the six software modules in detail. Chapter 5 reports the results obtained to date, including real-hardware validation, an automated test-suite summary, and adversarial testing of the authorization gate, and discusses these results against the related work identified in Chapter 2. Chapter 6 concludes the report and sets out the scope of work remaining before the final project review.

A reader interested specifically in what has been built and proven, rather than in the full research framing, can read Chapters 4 and 5 directly; a reader interested in how this project

positions itself against existing tools and published work can start from Chapter 2. The chapters are nonetheless written to be read in order, since each later chapter's claims are stated against terms — the novelty claims of Chapter 3, the module names and figures of Chapter 4 — introduced in the chapter before it.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

This chapter surveys the twenty peer-reviewed and pre-print sources that ground PISA's design, organized by the layer of the system each informs: WiFi-level security assessment (Section 2.2), network and device discovery in enterprise/IoT contexts (Section 2.3), protocol-aware device fingerprinting (Section 2.4), CVE correlation and exploit scoring (Section 2.5), authorized exploit validation (Section 2.6), and the state of unified IoT audit tooling as a whole (Section 2.7). Section 2.8 then compares PISA directly against named prior tools, platforms, and research systems, and Section 2.10 draws the chapter together into the specific research gaps this project addresses.

2.2 WIFI SECURITY ASSESSMENT AND BEACON-LEVEL ANALYSIS

Vanhoef and Ronen's analysis of WPA3's Dragonfly handshake, known as Dragonblood, showed that even the current generation of WiFi encryption carries exploitable timing and cache side-channels that permit offline password recovery, and that downgrade attacks can force a WPA3-capable network back to the weaker WPA2 handshake [19]. This result is significant for PISA because it establishes that a network's advertised encryption capability in its beacon frame is not, by itself, a reliable indicator of security — a station that has silently been downgraded is a materially different risk than one that was never configured for WPA3 at all, which is why PISA's WiFi Security Posture Score (WSPS, detailed in Chapter 4) treats protocol downgrade as a distinct, trackable signal rather than folding it into a single static encryption-type score. Vanhoef's subsequent, broader study of IEEE 802.11 frame aggregation and fragmentation handling identified twelve design-level flaws affecting every WiFi device shipped since 1997, independent of which encryption protocol is in use [18], reinforcing that passive, protocol-level observation of a network carries genuine security information well beyond a simple "is it encrypted" check. Schepers et al. extended this line of work by showing that an access point's transmit-queue behaviour during power-save mode can be manipulated to bypass WPA2 and WPA3 encryption outright [14], and Gvozdenovic et al. conducted a systematic study of the pre-authentication phase of WiFi connection establishment specifically, concluding that management frames — beacons among them — leak substantial, exploitable information before a client ever associates with the network [5].

Taken together, these four studies form the direct justification for PISA's Module 0 design decision: that a WiFi network's security posture can and should be assessed from beacon frames alone, before any active association, authentication, or packet injection is attempted. None of the four, however, propose an aggregate scoring methodology — each documents a specific class of vulnerability rather than a mechanism for turning passively observed signals into a single, field-actionable grade. That synthesis step is PISA's own contribution, discussed in Chapter 3.

Each of these four studies maps onto a specific, traceable factor in the WSPS formula described fully in Section 4.4.1, rather than informing the design only in general terms. Vanhoef and Ronen's Dragonfly side-channel result [19] is the direct motivation for treating encryption type as the single highest-weighted factor in the formula, since it is the factor that most directly reflects which handshake class — and therefore which known weaknesses — a network is exposed to. Vanhoef's frame-aggregation study [18], which affects WiFi devices independent of encryption generation, supports scoring signal and channel characteristics as independent factors rather than folding everything into the encryption score alone, since a frame-level vulnerability is not mitigated by stronger encryption. Schepers et al.'s transmit-queue manipulation result [14] and Gvozdenovic et al.'s pre-authentication leakage study [5] jointly motivate treating Management Frame Protection (802.11w) support and WPS exposure as explicit, separately weighted factors rather than secondary detail, since both studies identify pre-association management-frame behaviour, not post-association traffic, as the exploitable surface — exactly the surface WSPS is scoped to observe.

2.3 NETWORK DISCOVERY AND THE ENTERPRISE IOT ATTACK SURFACE

Nguyen et al. proposed DIoT, a federated, self-learning system that detects compromised IoT devices from network-level traffic anomalies across many deployments without centralizing raw traffic [10]. DIoT is architecturally significant as a privacy-preserving detection design, but it requires continuous network visibility, a federated training infrastructure, and cloud connectivity that a field-portable, point-in-time assessment tool such as PISA does not assume. Churcher et al.'s survey of the enterprise IoT (E-IoT) attack surface [3] is used in this project primarily as evidence rather than as a design template: it confirms, across a wide range of deployed protocols and device classes, that device discovery, protocol enumeration, and vulnerability correlation remain disconnected activities in current security practice — precisely the fragmentation problem described in Chapter 1.

2.4 PROTOCOL-AWARE IOT DEVICE FINGERPRINTING

2.4.1 Traffic-Flow and Machine-Learning Approaches

Sivanathan et al. demonstrated that multi-class IoT device identification from flow-level traffic features can exceed 97% accuracy across 28 device types [15], establishing the practical accuracy ceiling that any non-machine-learning alternative, including PISA's, is implicitly measured against. The cost of that accuracy is a dependency on a labelled training corpus per device class, which does not generalize to a device model the system has never seen before — a limitation this project's protocol-behavioural approach is specifically designed to avoid. Habibi Lashkari et al. showed that lightweight, optimized "digital footprint" feature sets can identify devices under the kind of resource constraints a Raspberry Pi 4 operates under [6], which directly supports the feasibility of running fingerprinting logic on the same commodity hardware PISA targets, independent of whether that logic is ML-based or rule-based.

2.4.2 Deterministic and Protocol-Specific Fingerprinting

Dong et al. addressed devices that speak proprietary, undocumented protocols using key-block pattern matching rather than a learned classifier [4], demonstrating that deterministic, non-ML fingerprinting is viable in principle, though their technique is evaluated on a single protocol family rather than the multi-protocol surface (MQTT, CoAP, HTTP, RTSP) PISA targets. Ren et al.'s large-scale empirical study of more than fifteen thousand consumer IoT traffic flows across MQTT, HTTP, and CoAP [13] provides ground-truth behavioural diversity data that corroborates the premise behind PISA's Module 2: that distinct protocols carry distinguishable, manufacturer-specific behavioural signatures worth probing independently and then fusing.

Three further sources ground PISA's protocol-specific probe design directly. Soni and Singh's systematic vulnerability assessment of the MQTT protocol [16] catalogues authentication bypass, topic enumeration, and plaintext credential exposure as recurring weaknesses, informing exactly which MQTT behaviours PISA's probe checks for. Palmieri et al.'s empirical evaluation of MQTT broker deployments found that 83% ran with no authentication configured by default [11] — a statistic this project cites directly in its problem framing — and Hasan et al. catalogued known MQTT exploit patterns to motivate runtime detection rather than static auditing alone [7]. On the CoAP side, Amsuess et al.'s security analysis of CoAP and DNS-over-CoAP name resolution [1] identifies the plaintext,

connectionless `/.well-known/core` resource-discovery surface that PISA's CoAP probe specifically exercises.

None of P07 through P14 combine MQTT, CoAP, HTTP, and RTSP probing in a single deterministic engine, and none propose a method for fusing confidence across protocols when a device responds on more than one of them simultaneously, which is the specific contribution PISA makes at this layer, described fully in Chapter 4.

As with the WiFi-layer literature in Section 2.2, each protocol-specific source maps onto a named, concrete probe in Module 2 rather than informing the design only in general terms. Soni and Singh's catalogue of MQTT weaknesses [16] — authentication bypass, topic enumeration, plaintext credentials — is the direct source for what the MQTT probe checks on connection (CONNACK flags, keepalive negotiation) and on wildcard subscription (topic hierarchy exposure); Palmieri et al.'s no-authentication prevalence statistic [16] justifies checking authentication state as a first-class signal rather than an incidental one. Amsuess et al.'s CoAP security analysis [1] is the direct source for probing CoAP's `/.well-known/core` resource directory and block-transfer negotiation behaviour, rather than only checking whether port 5683 is open. For HTTP and RTSP, no single reviewed paper specifies a probe design as directly as the MQTT and CoAP literature does; this is reflected honestly in Chapter 4, where the HTTP and RTSP probes are instead grounded in named, vendor-specific behavioural patterns (such as Hikvision's `/doc/page/login.asp` endpoint and Dahua's `/RPC2` JSON-RPC interface) rather than a single academic source, since protocol-behavioural fingerprinting for these two protocols specifically is closer to an engineering gap than a published one.

2.5 CVE CORRELATION AND EXPLOIT SCORING

Jacobs et al. introduced the Exploit Prediction Scoring System (EPSS), the first open, data-driven framework for predicting the probability that a given CVE will be exploited within the following thirty days, reporting an area-under-curve of 0.838 and updated daily by FIRST.org [9]. EPSS is a direct, load-bearing input to PISA's exploit scoring: it is the one of the three metrics PISA combines that reflects near-term, empirically observed attacker behaviour rather than a static severity rating. Subramaniam et al.'s longitudinal analysis found that EPSS scores are frequently low or entirely unavailable at the moment a CVE first enters the CISA KEV catalogue of confirmed, actively exploited vulnerabilities [12] — a result that is, on its face, a limitation of EPSS, but which this project reads instead as direct evidence

that no single metric (CVSS, EPSS, or KEV listing alone) is sufficient for prioritization, which is precisely why PISA computes all three and combines them rather than relying on any one in isolation.

Radoglou-Grammatikis et al. compared CVSS, EPSS, the Common Weakness Scoring System, and IoT-specific scoring frameworks, and found that none of the existing frameworks is designed for real-time, portable assessment [12], while Anand et al. demonstrated the technical feasibility of graph-based IoT risk scoring against live NVD feeds, though their design assumes cloud-scale compute rather than an edge device [2]. PISA adapts the real-time-feed principle from the latter while deliberately keeping the scoring computation itself light enough to run on a Raspberry Pi 4 without a cloud dependency.

2.6 EXPLOIT VALIDATION AND AUTHORIZATION

Heiding et al. deployed a practical IoT risk assessment framework, SAFER, across three multinational organizations and, in doing so, established an authorization-gate methodology as both a legal and an operational necessity before any exploit verification step is allowed to run [8]. This is the single most directly adopted piece of prior work in PISA's design: Module 4's mandatory named-operator confirmation and five-second countdown before any RouterSploit action becomes clickable (detailed in Chapter 4) is a direct, software-enforced implementation of the authorization-gate pattern Heiding et al. describe at an organizational-process level.

2.7 UNIFIED IOT SECURITY AUDIT TOOLING

Varga et al. conducted the most comprehensive recent review of IoT security audit tooling available at the time of this survey, examining more than forty tools and proposing a layered (network, protocol, application) reference architecture [20]. Critically for this project's motivation, their review explicitly concludes that no existing tool integrates wireless-network-layer assessment with device-layer CVE correlation within a single, portable system — the most direct piece of published evidence supporting PISA's central novelty claim, and the paper this project's problem statement in Chapter 1 cites most heavily.

2.8 COMPARATIVE ANALYSIS OF EXISTING TOOLS AND PLATFORMS

Beyond the academic literature surveyed above, a number of widely used practitioner tools occupy parts of the same problem space PISA addresses. Table 2.1 positions PISA

against the closest of these by the specific capability each does or does not provide, extending the capability-gap framing introduced in Table 1.1 with named, concrete alternatives.

The tools in Table 2.1 fall into three broad categories, and PISA's relationship to each differs. The first category — Pwnagotchi, Kismet, WiGLE — captures WiFi-layer data passively, the same capture discipline Module 0 uses, but stops at logging or crowdsourced mapping rather than computing an actionable security score; PISA's contribution relative to this category is specifically the WSPS scoring layer described in Section 4.4.1, not the capture technique itself, which is methodologically similar. The second category — WiFi Pineapple, Bettercap, RouterSploit — performs active, operator-driven assessment or exploitation, closer to what PISA's later modules do, but each requires an operator to already know which device or network to target and supplies no automated discovery-to-CVE pipeline of its own; PISA's Module 4 deliberately reuses RouterSploit as an execution backend rather than re-implementing exploit modules, so this is a relationship of composition, not displacement. The third category — Nmap and Shodan — performs discovery and indexing at, respectively, local-network and internet scale, and both are used or referenced within PISA's own pipeline (Nmap directly, as Module 1's scan engine; Shodan as the closest conceptual analogue to Module 0's OUI-to-CVE correlation, performed instead on local, non-internet-facing router hardware).

Table 2.1: PISA compared against existing tools and platforms

Tool / Platform	What it does	What it does not do, relative to PISA
Pwnagotchi	Passive PMKID capture and wardriving on Raspberry Pi Zero hardware	No CVE correlation, no security scoring, no device-layer assessment, not integrated with a cloud or reporting layer
Kismet	Passive WiFi (and other RF) logging across many protocols	Logs raw observational data only; no posture scoring, no CVE correlation, no device fingerprinting
WiGLE	Crowdsourced mapping of SSIDs and encryption types	Cloud-only aggregation; no per-session security scoring, no CVE correlation, no device layer
WiFi Pineapple (Hak5)	Commercial active WiFi auditing hardware platform	Proprietary hardware, no CVE correlation pipeline, not built for academic reproducibility
Bettercap	General-purpose network attack and MITM toolkit	Manual, operator-driven operation; no CVE pipeline, no device fingerprinting, not a portable standalone unit
RouterSploit	IoT/router exploitation framework with a large module index	No discovery stage, no fingerprinting, no automatic CVE

		lookup; PISA uses RouterSploit as its Module 4 execution backend rather than competing with it
Nmap (+ NSE scripts)	Network/port/service scanning and OS fingerprinting	No protocol-behavioural analysis beyond banner grabbing, no CVE pipeline; PISA uses Nmap as its Module 1 discovery backend
Shodan	Internet-scale indexing of internet-facing devices	Cloud-only, passive, no local/offline assessment, no exploit-verification pipeline

A recurring pattern in Table 2.1 is that PISA does not seek to replace Nmap or RouterSploit — both remain best-in-class at their specific function and are used internally as PISA's Module 1 and Module 4 execution backends respectively, as detailed in Chapter 4. PISA's contribution is the pipeline connecting discovery, fingerprinting, CVE correlation, and gated verification into one evidence-preserving chain, a role none of the tools in Table 2.1 individually fill.

2.9 CONSOLIDATED VIEW OF THE REVIEWED LITERATURE

Table 2.2 consolidates all twenty sources discussed in Sections 2.2 through 2.7 into a single reference, cross-indexed against the PISA module each most directly informs and the reference number it carries in this report's bibliography, to make the mapping between the literature review and the system design in Chapter 4 explicit and easy to verify.

Table 2.2: Consolidated summary of the twenty reviewed sources

Ref.	First author	Year	Venue	PISA module informed
[19]	Vanhoef & Ronen	2020	IEEE S&P	M0 — WPA3 downgrade awareness
[18]	Vanhoef	2021	USENIX Security	M0 — frame-level WiFi vulnerability
[14]	Schepers et al.	2023	USENIX Security	M0 — beacon/queue anomaly basis
[5]	Gvozdenovic et al.	2022	IEEE CNS	M0 — pre-authentication WiFi vulnerability
[10]	Nguyen et al.	2023	IEEE IoT Journal	M1 — compromised-device detection context
[3]	Churcher et al.	2021	IEEE Access	M1 — enterprise IoT attack-surface evidence
[15]	Sivanathan et al.	2020	arXiv	M2 — ML fingerprinting accuracy ceiling
[4]	Dong et al.	2023	Elsevier Computer Networks	M2 — deterministic, non-ML fingerprinting feasibility
[6]	Habibi Lashkari et al.	2022	arXiv	M2 — lightweight, edge-feasible fingerprinting

[13]	Ren et al.	2024	arXiv	M2 — cross-protocol behavioural ground truth
[16]	Soni & Singh	2021	IEEE ICCCA	M2 — MQTT probe design basis
[11]	Palmieri et al.	2021	ACM WiNTECH	M2 — MQTT no-auth prevalence statistic
[7]	Hasan et al.	2022	MDPI Sensors	M2 — MQTT exploit-pattern catalogue
[1]	Amsuess et al.	2023	ACM SIGCOMM	M2 — CoAP resource-discovery probe design
[9]	Jacobs et al.	2021	ACM DTRAP	M3 — EPSS exploit-prediction scoring
[17]	Subramaniam et al.	2024	arXiv	M3 — tri-metric scoring justification
[12]	Radoglou-Grammatikis et al.	2023	MDPI Electronics	M3 — scoring-framework landscape
[2]	Anand et al.	2023	Elsevier Computer Networks	M3 — real-time NVD feed feasibility
[8]	Heiding et al.	2020	arXiv	M4 — authorization-gate methodology
[20]	Varga et al.	2024	Springer IJIS	All — central unified-tool-gap evidence

2.10 SUMMARY OF RESEARCH GAPS

The twenty sources reviewed in this chapter collectively confirm five distinct research gaps that this project addresses directly:

- **G1 — No unified WiFi-and-device assessment tool.** Established most directly by the layered-architecture review [20] and the enterprise IoT attack-surface survey [3].
- **G2 — No real-time, EPSS/KEV-integrated exploitability scoring on edge hardware.** Established by [2], [9], [12], [17].
- **G3 — No passive, beacon-only WiFi posture grading framework.** Established by the WiFi-layer vulnerability studies [5], [14], [18], [19].
- **G4 — No non-machine-learning, cross-protocol device fingerprinting technique.** Established by [4], [6], [13], [15].
- **G5 — No cross-protocol confidence-fusion mechanism when a device is identifiable on more than one protocol at once.** Established by the single-protocol scope of [1], [7], [11], [16].

Chapter 3 formalizes these five gaps into the specific novelty claims and research questions this project sets out to answer, and Chapter 4 describes, module by module, how PISA's architecture answers each of them.

CHAPTER 3

PROBLEM STATEMENT AND METHODOLOGY

3.1 PROBLEM DEFINITION

Chapters 1 and 2 established that IoT security assessment today requires an assessor to manually operate four or more disconnected tools across two layers — WiFi-network security and device-level security — with no shared model of "the target" carried between them, and that no published tool or system closes this gap on portable, field-deployable hardware. This chapter restates that gap as a formal problem statement, derives concrete research questions and hypotheses from it, states the project's methodology, articulates PISA's five specific novelty claims against the related work surveyed in Chapter 2, summarizes the functional and non-functional requirements the system is built against, and sets out the ethical and legal boundaries the system is designed to operate within.

Formally: **given a wireless network and the IoT devices reachable on it, no existing portable system can (a) grade the network's own security posture from passive observation alone, (b) identify the specific devices on it without a labelled training corpus, (c) correlate those devices to real, currently-exploited vulnerabilities rather than keyword-matched ones, and (d) safely and verifiably confirm exploitability before reporting a device as at risk — all within a single pipeline on commodity edge hardware.** PISA is the system built to close this gap.

3.2 RESEARCH QUESTIONS AND HYPOTHESES

The primary research question this project investigates is whether a portable, self-contained system can perform end-to-end IoT security assessment — from wireless network vulnerability scoring through protocol-aware device fingerprinting to CVE-exploit correlation — without machine learning, on commodity hardware. This is decomposed into three supporting questions, each paired with a testable hypothesis evaluated in Chapter 5.

- **RQ2 (Network layer).** Can passive WiFi beacon-frame analysis alone produce a security-posture score that meaningfully predicts a network's exploitability, without active packet injection or authentication? — **H1:** the WSPS framework will assign security grades that agree with manual expert assessment on the same networks.
- **RQ3 (Device layer).** Do IoT devices from the same manufacturer and firmware generation exhibit sufficiently consistent protocol-behavioural signatures — MQTT

topic structure, CoAP response-option ordering, HTTP header ordering — to enable deterministic, non-ML device identification at accuracy comparable to ML-based approaches? — **H2**: protocol-aware behavioural fingerprinting will correctly identify device type with accuracy competitive with the ML baselines surveyed in Chapter 2.

- **RQ4 (Integration)**. What accuracy, completeness, and time-to-assessment does the integrated two-layer pipeline achieve relative to a manual multi-tool workflow (Nmap, Bettercap, RouterSploit, and manual NVD search)? — **H3**: the automated CVE-correlation pipeline will surface applicable high-severity CVEs faster than an equivalent manual search; **H4**: end-to-end assessment time on the integrated pipeline will be substantially lower than the equivalent manual workflow.

Table 3.1 states, for each hypothesis, the specific metric and target this project treats as evidence for or against it, so that the discussion in Chapter 5 can be read against a stated bar rather than an implicit one. Several of these evaluations require the physical hardware acquisition scoped in Chapter 6 and are reported in Chapter 5 only where real evidence toward them already exists.

Table 3.1: Planned evaluation metrics per hypothesis

Hypothesis	Metric	Target
H1 — WSPS validity	Agreement between WSPS grade and independent manual expert assessment on the same networks	High agreement across a multi-network sample
H2 — Fingerprinting accuracy	Per-protocol and fused-confidence precision/recall against known device ground truth	Accuracy competitive with the ML baseline reported in Section 2.4
H3 — CVE surfacing completeness	Time from device identification to a ranked, applicable CVE list; completeness against vendor-advisory ground truth	All applicable high-severity (CVSS ≥ 7.0) CVEs surfaced within a short, field-practical window
H4 — End-to-end timing	Total assessment time, PISA's integrated pipeline vs. an equivalent manual Nmap + Bettercap + RouterSploit + manual-NVD-search workflow	Substantial reduction relative to the manual baseline

3.3 DEVELOPMENT METHODOLOGY

PISA was developed iteratively rather than against a single fixed specification produced up front. Each module (M0 through M5) was specified in its own section of the project's Software Requirements Specification, implemented, covered by automated unit and integration tests, and only then marked **Implemented** in the specification's own requirement-

traceability table — a discipline that was maintained across seven tracked specification revisions as the system grew from WiFi-only scoring (M0) through network discovery (M1), protocol fingerprinting (M2), CVE correlation (M3), and gated exploitation (M4). This iterative, test-driven approach was chosen deliberately over a waterfall design-then-build process for two reasons specific to a security tool: first, several design assumptions — such as whether a Scapy-based ARP sweep would scale to a real, busy subnet — could only be validated against real hardware and real networks, not on paper; second, keeping every requirement explicitly labelled **Implemented** or **Planned** at all times (rather than assumed complete) was adopted as a project-wide honesty discipline to prevent the documentation from silently drifting ahead of the code, a risk this report itself follows by distinguishing, in Chapters 4 and 5, what has been built and validated from what remains future work.

Each module's test suite was written and run continuously via GitHub Actions on every push, giving the project a regression safety net from the first module onward; by the state reported in Chapter 5, this had grown to 381 automated tests spanning all six implemented modules. Validation was further organized into two deliberately distinct evidence tiers, kept visibly separate throughout development: automated unit/integration tests running against mocked external services (for fast, deterministic CI runs), and a smaller set of explicit real-hardware, real-network, real-external-API validation passes, the results of which are reported honestly in Chapter 5 as what was actually observed, not what the design predicts.

Table 3.2 traces this iterative growth directly through the Software Requirements Specification's own revision history, each entry corresponding to one module or capability reaching the **Implemented** status referenced throughout this report.

Table 3.2: Software Requirements Specification revision history

Version	Change
1.0	Initial SRS, aligned to the first implemented slice: M0 (WiFi assessment), the database layer, and M5 (dashboard)
1.1	Added PMKID / EAPOL handshake capture (passive mode) to M0
1.2	M1 (network join + device discovery) implemented; the earlier synthetic demo-data mode removed in favour of live-hardware validation
1.3	M3 device CVE correlation (NVD lookup) implemented
1.4	M1 extended with mDNS-based device identification
1.5	M2 (protocol behavioural fingerprinting) implemented
1.6	M3 extended with EPSS and CISA KEV correlation and the tri-metric exploit score
1.7	M4 (authorized RouterSploit exploit verification) implemented

Several of these transitions were driven by a measured engineering risk rather than a planning assumption, and Table 3.3 records the risks that materially changed the implementation as they were actually encountered, alongside the mitigation adopted — a risk register kept honest by only including risks that were real enough to have already changed a design decision described in Chapter 4, rather than a generic, unverified list.

Table 3.3: Technical risks encountered and their mitigations

Risk	Where it surfaced	Mitigation adopted
Scapy-based ARP sweep cannot keep pace with reply volume on a large, busy subnet	M1 host discovery (Section 4.3)	Shelled out to the system <code>'arp-scan'</code> binary instead
Sequential Nmap scanning becomes a multi-hour bottleneck once host discovery surfaces hundreds of live hosts	M1 service/OS scan (Section 4.4.2)	Pooled, concurrent scanning bounded by <code>'config.NMAP_MAX_WORKERS'</code>
Reverse DNS does not resolve device names on the actual target network	M1 device labelling (Section 4.3)	Switched to mDNS-based device identification (SRS v1.4)
Folding a live CVE lookup into WSPS scoring would make scoring depend on network connectivity	M0 scoring design (Section 4.4.1)	CVE correlation kept as a separate, on-demand lookup (FR-3), preserving NFR-2
RouterSploit's wordlist loader and shell module fail outright on Python 3.13	M4 dependency setup	Pinned compatibility shims (<code>'setuptools<81'</code> , <code>'standard-telnetlib'</code>) in requirements.txt

3.4 NOVELTY CLAIMS

Against the five research gaps identified in Chapter 2 (Section 2.10), PISA makes five specific novelty claims. Each is stated here against its closest related work; the corresponding engineering design is described in Chapter 4, and its current validation status is reported in Chapter 5.

3.4.1 Claim 1 — WiFi Security Posture Scoring (WSPS)

A quantitative, letter-graded (A-F) security score computed for a wireless network from passive beacon-frame analysis alone, with no active association, authentication, or packet injection. Existing wardriving tools such as Kismet and WiGLE capture comparable raw data but apply no scoring; no reviewed paper defines a passive-beacon-only posture-scoring framework closing gap G3 [5], [14], [18], [19].

3.4.2 Claim 2 — OUI-to-CVE Infrastructure Correlation

Automatic correlation of a WiFi access point's hardware identity, derived from its BSSID's Organizationally Unique Identifier (OUI), to applicable CVEs from the live NVD

database, computed on-demand rather than from a static offline list. Internet-facing device indexes such as Shodan perform a comparable correlation at internet scale on internet-facing devices; no reviewed tool performs it for local, non-internet-facing router hardware from passively captured beacon data.

3.4.3 Claim 3 — Non-ML Cross-Protocol Behavioural Fingerprinting

Device identification from *how* a device behaves within MQTT, CoAP, HTTP, and RTSP — topic hierarchy, response-option ordering, header ordering, and authentication-challenge format — fused across protocols when a device answers on more than one, using a probabilistic complement-fusion rule (combined confidence = $1 - \prod(1 - c_i)$ across each protocol's individual confidence c_i), rather than requiring a labelled training corpus per device class as the closest ML baseline does [15]. No reviewed single-protocol fingerprinting work [4], [6], [13] proposes a cross-protocol fusion mechanism, closing gaps G4 and G5.

3.4.4 Claim 4 — Integrated Two-Layer Pipeline on Portable Hardware

Chaining WiFi-layer assessment (Claims 1-2) with device-layer fingerprinting and CVE correlation (Claims 3, 5) in one automated pipeline on Raspberry Pi 4 class hardware, eliminating the fragmented Nmap-Bettercap-RouterSploit-manual-NVD-search workflow with a single field-deployable device, closing gap G1 [3], [20].

3.4.5 Claim 5 — Tri-Metric Exploitability Scoring at the Edge

Computing device-specific exploitability from all three authoritative signals together — CVSS v3.1 severity, FIRST.org's EPSS 30-day exploitation-probability score, and CISA KEV confirmed-exploitation listing — on edge hardware rather than requiring cloud-scale compute, closing gap G2. No reviewed portable IoT assessment tool combines EPSS with CVSS and KEV at the point of assessment [2], [9], [12], [17]; the full combination formula is given in Chapter 4.

3.5 REQUIREMENTS SPECIFICATION

PISA's Software Requirements Specification (conforming to the IEEE 830-1998 structure) defines twelve functional requirements and seven non-functional requirements against the system's v1 scope. Table 3.4 and Table 3.5 summarize both, each tagged by its current implementation status; the file(s) implementing each requirement are given in Chapter 4 alongside the module they belong to.

Table 3.4: Functional requirements summary (v1)

ID	Requirement	Module	Status
FR-1	WiFi beacon frame capture (BSSID, SSID, channel, signal, encryption, PMF, WPS)	M0	Implemented
FR-2	WiFi Security Posture Score (WSPS), 0-100 and A-F grade	M0	Implemented
FR-3	OUI-to-vendor resolution and on-demand vendor-to-CVE (NVD) lookup	M0	Implemented
FR-4	Scan session creation, status tracking, and persistence	M0 / DB	Implemented
FR-6	Web dashboard — session list, scan trigger, results, CVE lookup	M5	Implemented
FR-7	Network join + device discovery (ARP sweep, Nmap -sV -O, mDNS ID)	M1	Implemented
FR-8	Protocol behavioural fingerprinting (MQTT/CoAP/HTTP/RTSP) with fusion	M2	Implemented
FR-9	Device CVE correlation, enriched with EPSS + CISA KEV + exploit score	M0 / M3	Implemented
FR-10	Authorized exploit verification via RouterSploit, with audit logging	M4	Implemented
FR-12	PMKID / EAPOL handshake capture (passive mode)	M0	Implemented, passive only
FR-11	Cloud reporting — AWS sync and automated PDF generation	AWS	Planned

Table 3.5: Non-functional requirements summary (v1)

ID	Requirement	Status
NFR-1	A scan shall not block the dashboard from serving other requests	Implemented
NFR-2	WSPS scoring shall be a pure function of packet data, testable offline	Implemented
NFR-3	A failed capture shall mark the session errored, not crash the server	Implemented
NFR-4	The dashboard shall render with no external CDN/internet-hosted asset	Implemented
NFR-5	All SQLite writes shall enforce foreign-key constraints	Implemented
NFR-6	The automated test suite shall run in CI on every push/PR	Implemented
NFR-7	Exploit verification shall require logged, explicit operator authorization	Implemented

As Table 3.4 shows, every v1 functional requirement except cloud reporting (FR-11) is implemented; FR-5 (an early synthetic demo-data mode) was deliberately removed in v1.2 once live-hardware operation was considered the project's standard of evidence, a decision explained further in Chapter 4.

3.6 ETHICS, LEGAL CONSTRAINTS, AND SAFETY

PISA is designed exclusively for authorized security assessment, and this constraint shapes its architecture rather than being an afterthought layered on top of it. Under India's Information Technology Act, 2000, unauthorized access to a computer system is a civil

offence under Section 43 and, where accompanied by dishonest intent, a criminal offence under Section 66; PISA's design assumes use exclusively by a network's owner or an operator with explicit, documented authorization to test it, consistent with the authorization-gate methodology established in the organizational IoT risk-assessment literature [8] and discussed in Section 2.6.

Three design principles follow directly from this constraint and recur throughout the module designs in Chapter 4. First, the system never auto-exploits: Module 4's authorization gate structurally requires a named operator, an explicit confirmation, and a mandatory delay before any exploit action becomes available, regardless of how confident the system is in a finding. Second, every scan, join, fingerprint probe, and exploit attempt is written to an append-only audit log, not merely displayed and discarded, so that a record of what was done, by whom, and when exists independently of the dashboard session. Third, the dashboard binds to the local loopback interface by default rather than being exposed on the network — since its API has no authentication layer of its own and several of its endpoints accept WiFi passwords and trigger real network actions, exposing it by default would itself be a safety regression; an operator must explicitly opt in to exposing it, and is told exactly why in the system's own documentation.

A fourth principle governs how captured data itself is treated: WiFi passwords supplied for M1's network-join action exist only for the duration of the join operation and are not persisted in the session database alongside the rest of a network's record, and the passive PMKID/EAPOL material M0 can capture (Section 4.4.1) is stored as the hash material itself rather than in any plaintext-recoverable form, consistent with the principle that an assessment tool's own evidence store should not become a second attack surface. This principle is stated here as a binding design constraint that the implementation in Chapter 4 is held to, rather than as a claim about a specific encryption-at-rest mechanism not yet built in v1.

These constraints are revisited concretely in Chapter 4, where the specific state machine enforcing them in Module 4 is described, and in Chapter 5, where the gate is shown to have been tested adversarially — not merely asserted to exist.

CHAPTER 4

SYSTEM DESIGN AND IMPLEMENTATION

4.1 SYSTEM ARCHITECTURE OVERVIEW

PISA is organized as a single Flask application running six software modules — labelled M0 through M5 throughout this report, matching the project's own internal naming — on one machine. No module runs as a separate microservice and no module requires a cloud dependency for its core function; the only components that call out to the internet are the NVD, FIRST.org EPSS, and CISA KEV lookups in M3, all of which are triggered on demand rather than continuously. Figure 4.1 shows the five-layer pipeline this implies: the Hardware Layer (Raspberry Pi 4 plus an external monitor-mode WiFi adapter) feeds M0's WiFi assessment engine, whose output flags a target network for the M1-M2 discovery-and-fingerprinting stage, whose output in turn feeds the M3-M4 threat-intelligence-and-verification engine, with M5 presenting every stage's results through a single dashboard.

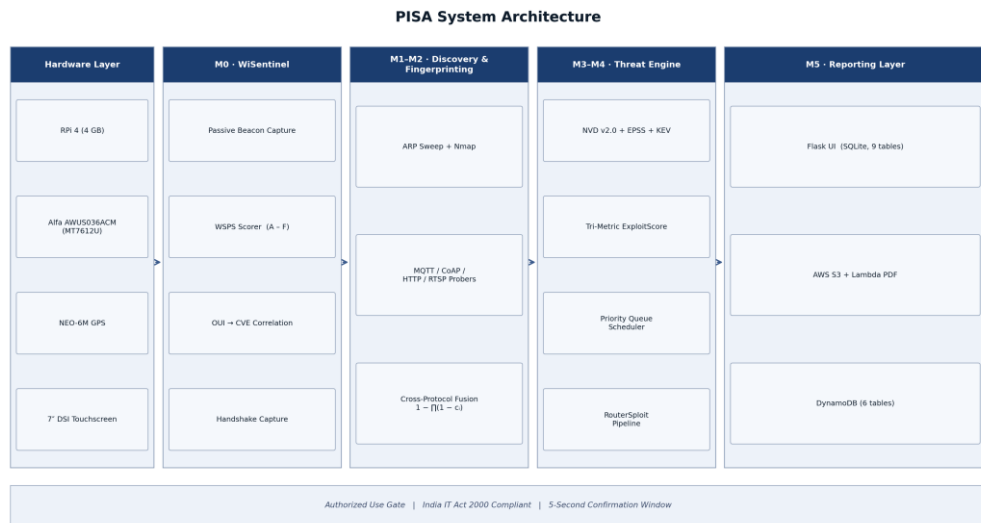


Fig. 1. PISA five-layer pipeline: M0 performs passive WiFi intelligence; M1-M2 execute network discovery and protocol fingerprinting; M3-M4 provide threat intelligence and exploit automation; M5 handles cloud reporting and dashboard.

Figure 4.1: PISA five-layer system architecture

Data flows strictly downstream in Figure 4.1 — a network must be assessed by M0 before it can be joined by M1, a device must be discovered by M1 before it can be fingerprinted by M2 or correlated against CVEs by M3, and M4 will not offer an exploit until M3 has returned an applicable finding for that specific device. This strict, one-directional dependency is a deliberate design choice, not an implementation accident: it is what makes the authorization gate described in Section 4.6 structurally enforceable rather than merely a

UI convention, since a later stage can never run ahead of the evidence an earlier stage actually established.

4.2 HARDWARE ARCHITECTURE

PISA's hardware target is a Raspberry Pi 4, chosen because it is inexpensive, has sufficient CPU headroom for the packet-processing and multi-threaded scanning workloads M0 through M3 require, and — critically — its built-in WiFi adapter (Broadcom BCM43455) can join a network in managed mode but does not expose monitor mode through its Linux driver. An external USB adapter is therefore required for M0's passive beacon capture; this project uses the Alfa AWUS036ACM (MediaTek MT7612U chipset), a dual-band 802.11ac adapter with a native mt76 kernel driver, chosen specifically because a 2.4 GHz-only adapter would miss the many networks — most WPA3 deployments among them — that only broadcast on 5 GHz. The Raspberry Pi's own built-in adapter is then free to serve M1's network-join role (`config.JOIN_IFACE`), giving the system two independent radios: one permanently in monitor mode for passive capture, one in managed mode for active device discovery, so that joining a target network never interrupts the background WiFi assessment. Table 4.1 summarizes the hardware used to develop and validate the system reported in Chapter 5, alongside peripherals scoped for a later hardware-integration phase but not yet part of v1.

Table 4.1: Hardware specification

Component	Role	Status in v1
Raspberry Pi 4 (4 GB)	Core compute platform; runs the Flask application and all six modules	Core — used for validation
Alfa AWUS036ACM (MT7612U)	Dual-band 802.11ac monitor-mode adapter for M0 beacon capture	Core — used for validation
Built-in WiFi (BCM43455)	Managed-mode radio for M1 network join, independent of the monitor-mode adapter	Core — used for validation
ESP32 development board (firmware/esp32-iot-target)	Controlled IoT test target running MQTT/HTTP services for M2/M3 validation	Core — used for validation
NEO-6M GPS module + USB-UART adapter	Geolocation tagging of assessment sessions (warwalking)	Planned extension — not yet integrated
7-inch DSI touchscreen	Standalone field UI, removing the dependency on a separate laptop/phone browser	Planned extension — not yet integrated

A deliberate hardware simplification follows from the radio arrangement described above: an earlier design draft specified a second purchased USB WiFi adapter for the

managed-mode join role, until it was confirmed that the Raspberry Pi's own built-in adapter already covers that role, removing one line item without any loss of capability. Table 4.2 gives an indicative cost breakdown for the hardware a new build of PISA requires beyond a Raspberry Pi 4 already on hand, separating what v1's validation in Chapter 5 actually used from the planned extensions in Table 4.1.

Table 4.2: Indicative hardware cost (components not already owned)

Component	Indicative cost (INR)	Required for
Alfa AWUS036ACM monitor-mode adapter	~7,000	Core v1 (M0)
ESP32 development board + sensors	~800	Core v1 (M2/M3 controlled test target)
Powered USB hub (self-powered, 4-port USB 3.0)	~800	Core v1 (stable Alfa power delivery)
NEO-6M GPS module + USB-to-UART adapter	~800	Planned extension (geolocation tagging)
7-inch DSI touchscreen	~6,600	Planned extension (standalone field UI)

The self-powered USB hub in Table 4.2 is a deliberate, non-obvious inclusion rather than an incidental accessory: the Alfa adapter draws enough current under active scanning that a bus-powered hub was found to cause it to drop or behave unstably, so the hub's own power supply, not just its port count, is the reason it is specified.

4.3 SOFTWARE ARCHITECTURE AND TECHNOLOGY STACK

The entire system is written in Python 3.12 for consistency across modules and native compatibility with Raspberry Pi OS. Table 4.3 lists the libraries actually pinned in the project's `requirements.txt` and the module each supports, rather than a larger aspirational list — for example, no machine-learning, Modbus, or PDF-generation library is present, consistent with those being out of v1 scope as stated in Chapter 1.

Table 4.3: Core software dependencies (requirements.txt)

Library	Version pinned	Used by
Flask	3.0.3	M5 — dashboard, REST API endpoints
Scapy	2.5.0	M0 — raw 802.11 beacon capture and parsing
python-nmap	0.7.1	M1 — Nmap service/OS scan wrapper
paho-mqtt	2.1.0	M2 — MQTT behavioural probe
aiocoap	0.4.7	M2 — CoAP behavioural probe
requests	2.31.0	M0/M3 — NVD, FIRST.org EPSS, CISA KEV HTTP calls; M2 HTTP/RTSP probes

boto3	1.34.131	AWS SDK — present for the planned M-AWS cloud-reporting stub
routersploit	3.4.7	M4 — exploit module index and execution backend
standard-telnetlib, setuptools<81	3.13.0 / n/a	Python 3.13 compatibility shims required by RouterSploit
arp-scan, nmap (system binaries)	n/a	M1 — shelled out to for ARP host discovery and OS/service scanning

Two dependencies in Table 4.3 were specifically chosen over a more "natural" Python-native alternative after that alternative was tried and measured against real conditions, a pattern that recurs across this project and is reported honestly rather than smoothed over: M1 shells out to the system `arp-scan` binary rather than performing a hand-rolled Scapy ARP sweep, because Scapy's Python-level reply matching could not keep pace with reply volume on a real, busy campus subnet of roughly eight thousand addresses, surfacing only a handful of hosts where `arp-scan` completed a full sweep of the same subnet in about thirty seconds; and M1's device-type labelling uses mDNS rather than reverse DNS, after reverse DNS was confirmed non-functional against the project's actual target network while mDNS was confirmed to resolve real device names independently of that network's own DNS infrastructure.

4.4 MODULE DESIGN

4.4.1 M0 — WiFi Assessment Layer

M0 captures 802.11 management frames (type 0, subtype 8 — beacon frames) from the monitor-mode adapter using raw Scapy sockets, filtering and parsing SSID, BSSID, channel, signal strength, RSN information element (encryption type and Protected Management Frame capability), beacon interval, and WPS vendor-specific information element presence, deduplicating by BSSID and persisting each result to the `networks` table. From this packet-derived data alone, M0 computes the WiFi Security Posture Score (WSPS): a 0-100 score built additively from seven factors, shown in Table 4.4, clamped to the [0, 100] range and mapped to a letter grade.

Table 4.4: WSPS scoring factors (pisa/m0/wsps.py)

Factor	Points	Scoring logic
Encryption type	up to 35	WPA3 = 35, WPA2 = 25, WPA = 10, Open = 0
Channel	up to 15	Non-overlapping channel (1/6/11) = 15, other = 5, none = 0
Signal strength	up to 20	> -50 dBm = 20, > -70 dBm = 12, > -85 dBm = 6,

		weaker = 0
Beacon interval	up to 12	Standard 100 TU = 12, non-standard = 5
Management Frame Protection (802.11w)	+15	PMF capability bit set in RSN IE = +15, else 0
WPS status	-15	WPS vendor IE present = -15 penalty, absent = 0
Hidden SSID	-10	Empty SSID in beacon = -10 penalty, else 0

Grades are assigned as A (score ≥ 90) down to F (score < 30). Two factors originally scoped — pairwise cipher-suite strength (CCMP versus TKIP) and folding router CVE exposure directly into the score — were deliberately deferred rather than silently dropped. The cipher-suite factor needs no new capture logic (the RSN information element already parsed for encryption-type detection carries the AKM suite) and is a natural near-term addition. CVE exposure, however, was deferred for an architectural reason: folding a live NVD lookup into `score\_network()` would make WSPS scoring depend on network connectivity, breaking the property that scoring is a pure function of already-captured packet data (Non-Functional Requirement NFR-2, Table 3.2) — a property the module's own unit tests rely on to run entirely offline. Vendor CVE exposure is instead surfaced as a separate, on-demand lookup (FR-3): M0 resolves the access point's manufacturer from its BSSID's OUI against the IEEE OUI database and, only when the operator explicitly requests it from the dashboard, queries the live NVD 2.0 API for CVEs associated with that vendor.

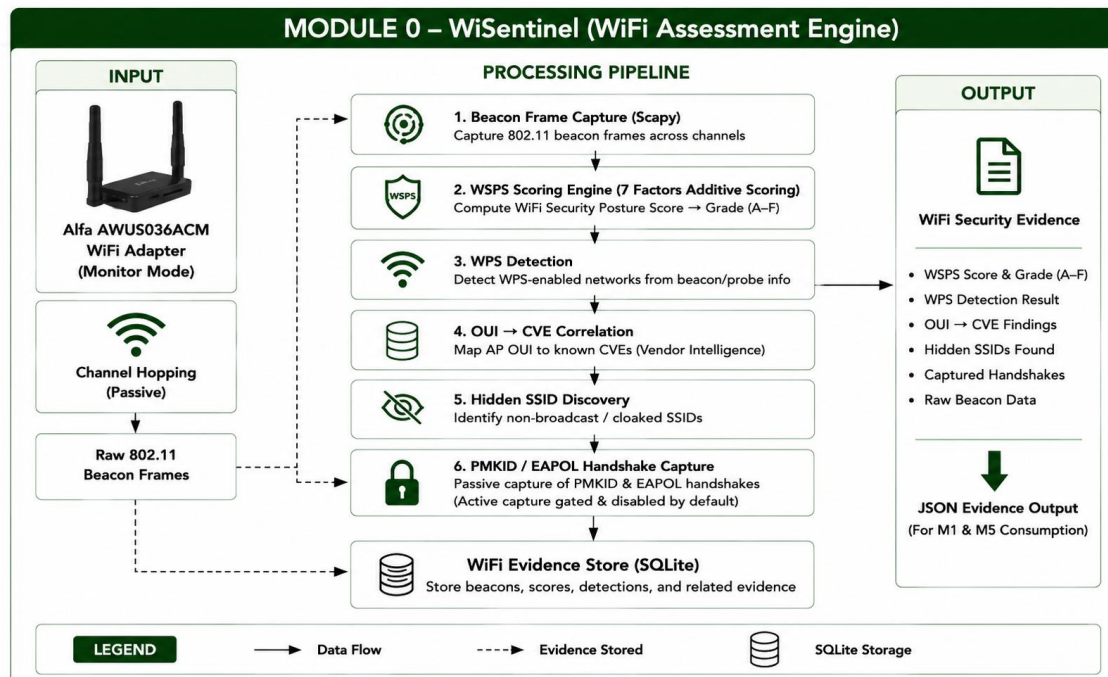


Figure 4.2: M0 (WiSentinel) internal processing pipeline

Figure 4.2 expands the single "M0 — WiFi" box of Figure 4.1 into its internal stages: beacon capture feeds the WSPS scoring engine, WPS detection, and OUI-to-CVE correlation in parallel, with every stage's output written to a dedicated WiFi evidence store rather than held only in memory, so that a network's score, its WPS/hidden-SSID findings, and any CVE correlation already performed for it survive independently of the dashboard session that triggered them. The PMKID/EAPOL handshake-capture stage shown in the figure is passive by default and deliberately gated when operated in its active (deauth-based) form, consistent with the authorization-gate principle introduced in Section 3.6 and implemented concretely for exploit verification in Section 4.6.

4.4.2 M1 — Network Discovery

On operator request, M1 joins a network already scored by M0 using its supplied password, via `nmcli` on the managed-mode interface, then builds a device inventory of the joined subnet in two stages: an `arp-scan` sweep for live hosts (IP, MAC, OUI-resolved vendor), followed by a concurrent Nmap `-sV -O` scan across a thread pool, restricted to common IoT-relevant ports (1883/MQTT, 5683/CoAP, 502/Modbus, 554/RTSP, plus standard web/management/remote-access ports) for open ports, service banners, and an OS guess. Each host is additionally queried over mDNS (`224.0.0.251:5353`) for a self-announced friendly name and service type. Running the Nmap stage concurrently, rather than one host at a time, was a direct response to a measured bottleneck: once the `arp-scan` fix described in Section 4.3 started surfacing hundreds of live hosts on a real subnet instead of a handful, a sequential scan became a multi-hour operation; a thread pool bounded by `config.NMAP_MAX_WORKERS` brought this back to a field-practical duration. A single host's scan failure (an Nmap crash, a vendor-lookup error) is isolated and recorded as a warning-severity alert rather than aborting the hosts already scanned successfully elsewhere in the pool.

Each discovered host's open-port set is the handoff point into Module 2: Table 4.5 shows the routing rule that maps an open port to the protocol probe M2 runs against it, with CoAP probed unconditionally on every device for the reason given in Section 4.4.3.

Table 4.5: M1-to-M2 port-to-protocol routing rule

Open port (from M1)	Protocol probe triggered (M2)
1883	MQTT
80, 443, 8080, 8443	HTTP
554	RTSP

5683 (UDP, not in Nmap's TCP results)	CoAP — probed unconditionally on every device
More than one of the above open	All applicable probes run; results merged by Cross-Protocol Confidence Fusion (Section 4.4.3)

4.4.3 M2 — Protocol-Aware Behavioural Fingerprinting

M2 probes each discovered device's open ports against the protocol conventionally associated with that port (MQTT on 1883, HTTP on 80/443/8080/8443, RTSP on 554), and probes CoAP (5683) unconditionally on every device regardless of Nmap's port list, since CoAP runs over UDP and never appears in Nmap's TCP-only open-port results from M1. Each protocol probe fails soft on any timeout or error — returning an empty result rather than raising — and, where it succeeds, returns one or more evidence features, each optionally suggesting a candidate device type. The mDNS-derived type already recorded by M1 is included as one such candidate at a fixed baseline weight. Panel M1-M2 of Figure 4.3 shows this routing and probing pipeline; panel M4's "Cross-Protocol Fusion" box shows where the per-protocol confidence scores are combined.

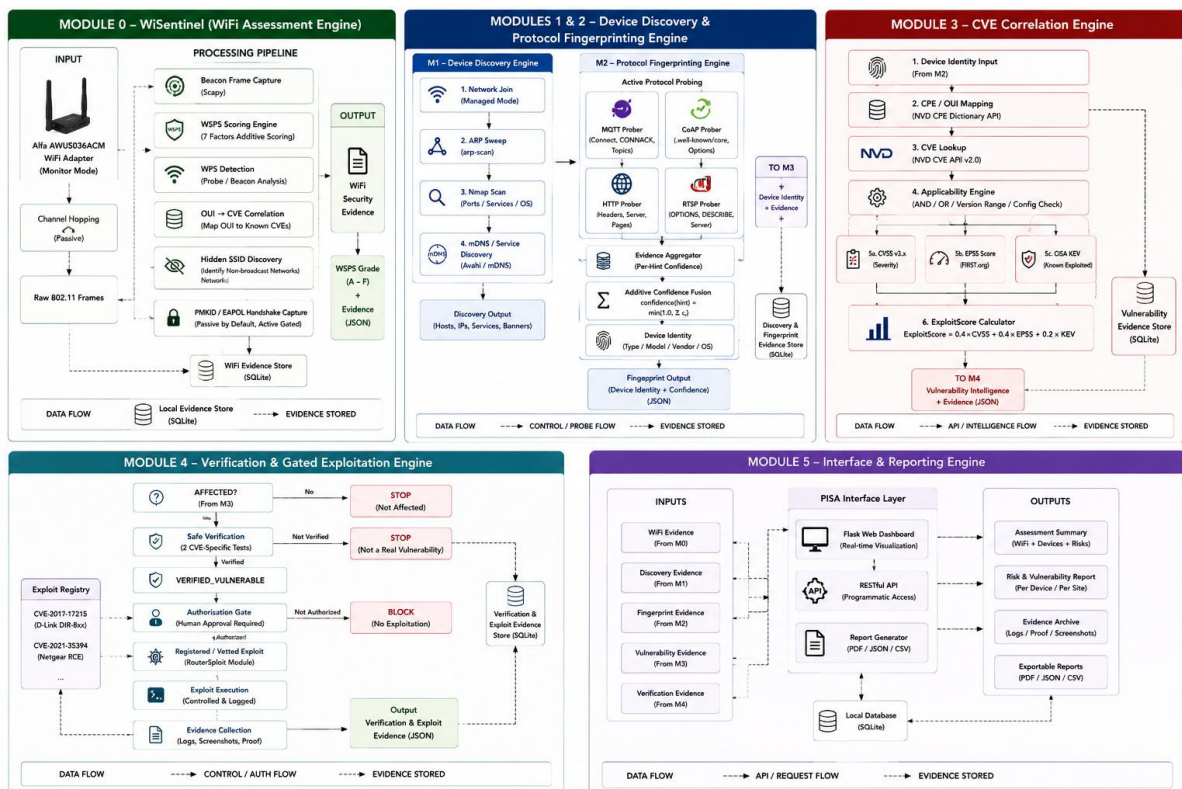


Figure 4.3: Per-module processing pipelines, M0 through M5

Fusion sums each candidate device-type's confidence across every protocol that proposed it, capped at 1.0, using a probabilistic complement rule rather than a simple average: two moderate-confidence signals (for example 0.75 from MQTT and 0.80 from

HTTP) combine to $1 - (1 - 0.75)(1 - 0.80) = 0.95$, so agreement across independent protocols raises confidence faster than either signal alone — the specific mechanism behind Novelty Claim 3 in Section 3.4. The highest-scoring candidate, with its fused confidence, is written back to the device's record. Devices can be fingerprinted individually or in bulk across a network via a thread pool (`config.M2_MAX_WORKERS`), for the same reason M1's Nmap stage is pooled: a mandatory CoAP probe alone costs up to `config.M2_PROBE_TIMEOUT` per device, making a sequential sweep impractical at realistic device counts.

Table 4.6 lists representative, concrete behavioural signatures each protocol probe checks for, to make Claim 3's "how a device behaves, not just which services it runs" distinction (Section 3.4.3) tangible rather than abstract.

Table 4.6: Representative behavioural signatures probed per protocol

Protocol	Behavioural signature probed	Example evidence
MQTT	CONNACK response flags, topic hierarchy on wildcard subscription, retained-message and keepalive handling	Topic pattern <code>'tele/{hostname}/STATE'</code> is characteristic of Tasmota-flashed devices
CoAP	<code>/.well-known/core</code> resource directory content, block-transfer negotiation, observe-notification interval	Resource path naming conventions specific to a given CoAP stack implementation
HTTP	Server header value, response header ordering, login-page form field names, error-response format	<code>'/doc/page/login.asp'</code> presence is characteristic of Hikvision camera firmware
RTSP	OPTIONS response headers, DESCRIBE/SDP format, authentication-challenge type (Digest/Basic/None)	<code>'Server: Dahua'</code> combined with a <code>'/RPC2'</code> JSON-RPC endpoint is characteristic of Dahua cameras

4.4.4 M3 — Vulnerability Intelligence

M3 correlates each fingerprinted (or, at minimum, OS-guessed) device against the live NVD CVE API, preferring the device's Nmap OS guess as the search keyword over its vendor alone — simplified to drop version-specific detail, since NVD's keyword search ANDs every token together and a literal version string in the query can zero out otherwise-relevant results. A vendor-only fallback was deliberately not implemented after it was found, in practice, to return the same decade-old, generically-matched CVEs regardless of the actual device; M3 returns an explicit "no data" result rather than a low-confidence guess in this case, consistent with the project's broader discipline of never reporting a finding it cannot actually support with evidence. Every CVE returned is enriched with its FIRST.org EPSS score

(probability of real-world exploitation within 30 days) and its CISA KEV listing status, then combined into a single 0-100 exploit score:

$$\text{exploit_score} = 0.4 \times \text{CVSS_normalized} + 0.4 \times \text{EPSS} + 0.2 \times \text{KEV_bonus}$$

where CVSS and EPSS are weighted equally, a CISA KEV listing contributes a flat 0.2 bonus, and a missing CVSS or EPSS value contributes zero to its own term rather than being excluded or renormalized — a deliberate choice that penalizes incomplete evidence instead of inflating a score around it. This formula operationalizes Novelty Claim 5 (Section 3.4.5) and is a direct application of the tri-metric principle established in the EPSS/KEV literature reviewed in Section 2.5 [9], [17]. Beyond scoring, M3 also implements the applicability engine that determines whether a CVE genuinely applies to a device — evaluating NVD's real AND/OR/version-range configuration logic against the device's identity, rather than treating any keyword match as a positive finding — and the live verification stage that must return a confirmed-vulnerable result before Module 4 will offer exploitation at all, detailed in Section 4.6.

To make this formula concrete rather than purely symbolic, consider the real reference finding reported in Chapter 5: CVE-2017-7577, returned by a live NVD lookup with CVSS = 9.8 and an EPSS score of 29% retrieved live from FIRST.org. If this CVE were not listed in the CISA KEV catalogue, its exploit score would be $0.4 \times 0.98 + 0.4 \times 0.29 + 0.2 \times 0 = 0.508$, or roughly 51 out of 100; were it additionally KEV-listed, the flat 0.2 bonus would raise the same finding to 0.708, roughly 71 out of 100 — a single confirmed real-world exploitation record moving a finding from the middle of the scoring range toward its upper end, independent of CVSS or EPSS changing at all. This is precisely the behaviour the tri-metric design in Novelty Claim 5 (Section 3.4.5) is intended to produce: KEV listing acts as a correction for exactly the case the literature identifies as EPSS's own blind spot [12] — a CVE whose EPSS score has not yet caught up with its observed real-world exploitation.

4.4.5 M4 — Authorized Verification and Gated Exploitation

M4 is scoped narrowly and deliberately: given a device and a CVE that M3 has already surfaced and verified live, M4 searches RouterSploit's module index (roughly 358 modules, of which only around two dozen cite an explicit CVE ID) for a matching exploit module. If a match exists, the dashboard exposes it only behind three gates in sequence — a named operator field ("authorized by"), an explicit confirmation checkbox, and a mandatory five-second countdown before the action becomes clickable — matching Non-Functional

Requirement NFR-7 (Table 3.5) and the authorization-gate pattern from the organizational risk-assessment literature [8] discussed in Section 2.6. "Check only" (RouterSploit's non-destructive `check()` method) is the default mode; "full exploit" (`run()`) is a separate, explicitly selected mode. Every attempt — check or run, success or failure — is written as its own row in the `exploit_results` table with the authorizing operator and both an authorization and an execution timestamp, as an append-only audit log rather than a cached, overwriteable lookup. The precise state machine enforcing all of this is described in Section 4.6 and is shown in Chapter 5 to have been tested adversarially rather than merely implemented.

The module-matching step deserves a closer look, since it is the specific mechanism that keeps M4 from ever executing an unvetted module. RouterSploit's module index is searched by CVE identifier, against a curated registry PISA itself maintains rather than RouterSploit's full, unfiltered module set — a deliberate narrowing, since not every module RouterSploit ships has been reviewed for safe behaviour against the specific device classes PISA targets. Coverage against this registry is inherently partial: with only around two dozen of RouterSploit's roughly 358 modules citing an explicit CVE ID at all, a CVE returning no match is an expected, honest outcome rather than a gap to be hidden, consistent with the project's broader discipline, established for M3 in Section 4.4.4, of never manufacturing a positive result where the evidence does not support one. This registry-based lookup is also what the adversarial test in row four of Table 5.3 (Chapter 5) specifically exercises: a client request that supplies its own `module_path` is checked against — and overridden by — this same server-side registry resolution, rather than being trusted directly.

4.4.6 M5 — Dashboard and Reporting

M5 is a Flask application that is simultaneously the operator's control surface and the only way any other module is actually triggered — none of M0 through M4 run autonomously in the background in v1; each is invoked on demand from a dashboard action or an equivalent CLI flag on `run.py`. The dashboard lists past sessions, starts new scans, displays per-session networks with WSPS grade badges, and — per network or device — exposes the on-demand CVE lookup, join-and-discover, fingerprint, and exploit-verification actions described above. By design it binds to `127.0.0.1` and must be explicitly reconfigured (`--host 0.0.0.0` or an environment variable) to be reachable from another device, since its API carries no authentication of its own and several of its endpoints accept WiFi passwords, consistent with the ethics and safety posture set out in Section 3.6.

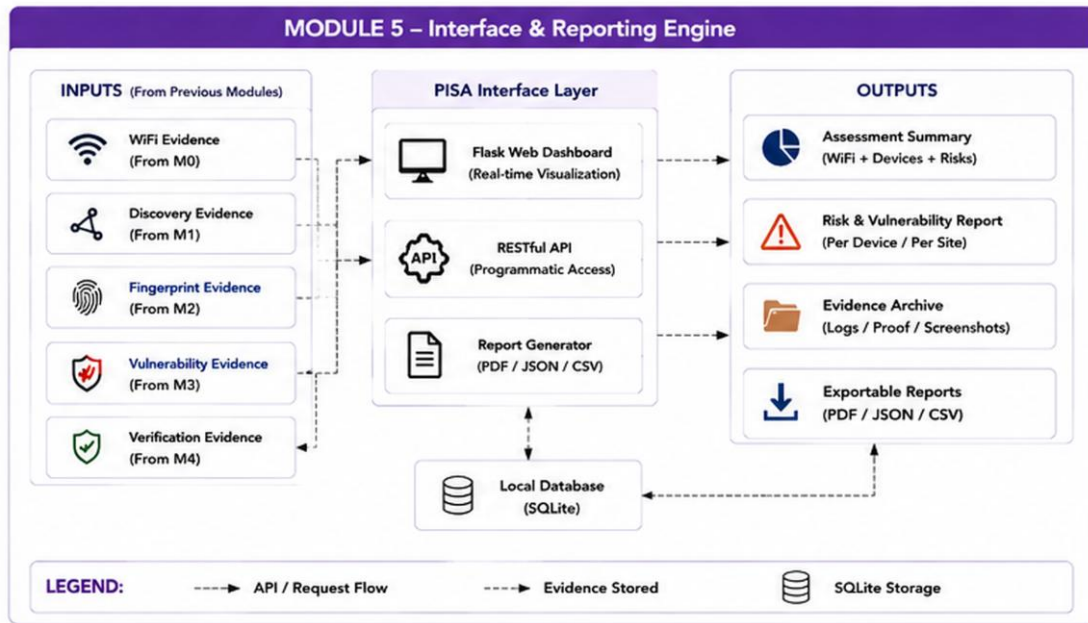


Figure 4.4: M5 interface and reporting engine

Figure 4.4 shows M5 as the convergence point for every evidence stream produced by M0 through M4: WiFi evidence, discovery evidence, fingerprint evidence, vulnerability evidence, and verification evidence are each written independently to the local SQLite database as they are produced, and the dashboard's web and REST interfaces read from that shared store rather than from any one module directly. The "Report Generator (PDF/JSON/CSV)" and "Evidence Archive" boxes in the figure correspond to the cloud-backed reporting capability scoped in Chapter 1 as planned future work (`pisa/aws/`); in the current release, session data is exported directly from the dashboard's existing JSON API rather than through a dedicated report-generation service.

4.5 DATABASE DESIGN

All assessment data is persisted locally first, in a single SQLite database, so that the core pipeline works fully offline except for the on-demand NVD/EPSS/KEV calls in M3; this is also why cloud sync (AWS) is additive future work rather than a dependency the current system requires to function. Table 4.7 summarizes the nine tables created on startup. A deliberate simplicity choice runs through the schema: it uses plain `INTEGER PRIMARY KEY AUTOINCREMENT` identifiers and no SQL-level `CHECK` constraints, because every constraint that matters is already enforced in the application's query layer, and SQLite's constraint checking would be redundant with it.

Table 4.7: SQLite schema (`pisa/db/models.py`)

Table	Purpose	Populated by
sessions	One row per assessment session; status and timing	M0
networks	Beacon-derived network records, WSPS score and grade	M0
network_cves	On-demand OUI-to-vendor CVE lookups per network	M0
devices	Discovered device records — IP, MAC, vendor, open ports, OS guess, fingerprint	M1 / M2
device_cves	Device-specific CVE correlation with CVSS/EPSS/KEV/exploit score	M3
exploit_results	Append-only log of every exploit authorization and outcome	M4
fingerprint_signatures	Per-protocol evidence features captured during M2 probing	M2
behavioral_drift	Session-to-session fingerprint delta records	Reserved — M2 extension
alerts	Unified alert log — scan failures today; broader use as modules extend	M0-M5

4.6 SECURITY AND AUTHORIZATION GATE DESIGN

The authorization gate described functionally in Sections 3.6 and 4.4.5 is implemented as an explicit state check in ``pisa/m4/routersploit_gate.py``, evaluated before any exploit action is permitted to run, encoded as a single boolean condition: ``applicability == AFFECTED AND verification == VERIFIED_VULNERABLE AND module is a specifically registered, vetted module AND an explicit named human authorization is present``. Each term in this condition corresponds to a distinct module's output — ``applicability`` from M3's configuration-tree evaluation, ``verification`` from M3's live verification test, the module registry from M4's own vetted list — so that no single module's output, on its own, is ever sufficient to unlock exploitation. This design directly operationalizes the "never conflate a CVE match with a confirmed vulnerability" principle stated as part of Novelty Claim 4: a keyword-only NVD match is structurally incapable of reaching the ``AFFECTED`` state this gate checks for, since ``AFFECTED`` is only reached after the applicability engine's AND/OR/version-range evaluation, not before it. Chapter 5 reports the results of testing this gate adversarially against the running dashboard, including a client attempting to supply an arbitrary exploit module path directly.

4.7 TESTING STRATEGY AND CONTINUOUS INTEGRATION

Every module is covered by its own automated test suite under ``tests/``, organized to mirror the module layout (``tests/m0/`` through ``tests/m5/``, plus ``tests/db/``), run with `pytest` and measured with branch coverage via `pytest-cov`. The suite runs in GitHub Actions on

every push and pull request against the main branch (`.github/workflows/test.yml`), giving the project a continuous regression safety net from the first module onward rather than a test pass performed once before a review. Tests that exercise external services — the NVD, EPSS, and KEV HTTP calls in M3, and RouterSploit's execution path in M4 — mock those calls so that CI remains fast and deterministic; this is deliberately kept separate from the real-hardware, real-network, real-API validation runs reported in Chapter 5, which exist specifically to confirm that what the mocked tests assert is actually true against live systems, not only internally consistent.

4.8 DEPLOYMENT AND OPERATION

Operating PISA requires the monitor-mode adapter to be placed into monitor mode before the dashboard is started — ``nmcli device set <iface> managed no``, followed by bringing the interface down, setting ``iw dev <iface> set type monitor``, and bringing it back up. The ``nmcli ... managed no`` step is not cosmetic: without it, NetworkManager silently reclaims the adapter back to managed mode the moment the link comes up, and a scan then completes with a ``done`` status, zero logged errors, and zero captured beacons — the channel-hopping and sniffing code runs without fault, it is simply listening on a managed-mode socket that never receives a raw 802.11 frame. This specific failure mode, discovered during development rather than assumed away, is documented in the project's own operating instructions precisely because it produces no error message of its own; an operator is told to check that ``iw dev <iface> info`` still reports ``type monitor`` if a scan finishes clean but empty, and to repeat the sequence after every reboot, since the interface reverts to managed mode on restart.

The dashboard itself must then be started as root, since ``iw``, Scapy's raw-socket beacon capture, and ``arp-scan`` all require ``CAP_NET_ADMIN`/`CAP_NET_RAW`` capabilities; the project runs the whole application as root rather than isolating only the specific calls that need the privilege, a deliberate simplicity trade-off for a tool whose field operator is assumed to already hold physical access to the hardware. Both the full dashboard (``sudo venv/bin/python run.py``) and a headless CLI path (``run.py --scan --iface <iface> --duration <seconds>``, or ``--join-network`/`--password`/`--join-iface`` for M1) are supported, so the system can be operated identically whether or not a browser is available on the field device — a design decision that anticipates, without yet implementing, the dedicated touchscreen interface scoped as future work in Chapter 6.

4.9 SUMMARY OF ENGINEERING DESIGN DECISIONS

Several of the design choices described across this chapter were not the first approach tried — each was arrived at after an earlier, more conventional choice was measured against real conditions and found wanting. Table 4.8 collects these decisions in one place as a single reference, since they are individually easy to lose track of when read across six separate module subsections, but collectively they are the clearest evidence that this system's design was shaped by what was actually observed during development, not only by what was planned before it started.

Table 4.8: Key engineering design decisions and their rationale

Decision	Rationale
`arp-scan` (shelled out) instead of a hand-rolled Scapy ARP sweep	Scapy's reply matching could not keep pace with reply volume on a real, busy subnet (Section 4.3)
mDNS instead of reverse DNS for device-type labelling	Reverse DNS was confirmed non-functional on the actual target network; mDNS was confirmed to work (Section 4.3)
Vendor CVE lookup kept separate from WSPS scoring	Keeps `score_network()` a pure, offline-testable function (NFR-2) rather than dependent on network connectivity (Section 4.4.1)
Concurrent, thread-pooled Nmap scanning in M1	Sequential scanning became a multi-hour bottleneck once `arp-scan` started surfacing hundreds of real hosts (Section 4.4.2)
A curated RouterSploit module registry rather than its full module set	Not every shipped module has been vetted for safe behaviour against PISA's target device classes (Section 4.4.5)
Whole-application root privilege rather than per-call privilege isolation	A simplicity trade-off appropriate for a tool whose operator already holds physical hardware access (Section 4.8)
Self-powered USB hub specified for the Alfa adapter	A bus-powered hub was found to cause the adapter to drop or behave unstably under active scanning (Section 4.2)

CHAPTER 5

RESULTS AND DISCUSSION

5.1 IMPLEMENTATION STATUS SUMMARY

This chapter reports results exactly as observed at the current review point, following the same discipline set out in Chapter 3: a result is reported as real only where it was obtained against live hardware, a live network, or a live external API, and is explicitly labelled where it is not. Table 5.1 summarizes the implementation status of every module introduced in Chapter 4 as a starting point for the detailed results that follow.

Table 5.1: Module implementation status at the current review

Module	Status	Evidence tier
M0 — WiFi assessment (beacon capture, WSPS, OUI-CVE)	Implemented	Real hardware
M1 — Network discovery (join, ARP sweep, Nmap, mDNS)	Implemented	Real network
M2 — Protocol behavioural fingerprinting	Implemented	Real, controlled target
M3 — CVE correlation (NVD + EPSS + CISA KEV + applicability)	Implemented	Real external APIs
M4 — Authorized verification and gated exploitation	Implemented (software)	Gate proven live; physical exploit target pending
M5 — Dashboard / API	Implemented	Real routes, live + demo mode
AWS cloud reporting (pisa/aws/)	Planned	Not started — intentional v1 stub

Every module proposed and scoped in Chapters 3 and 4 is implemented except cloud reporting, which remains an intentional stub unchanged since the project's initial scoping, as stated in Chapter 1. The remainder of this chapter substantiates each "Implemented" row with the specific evidence obtained for it.

5.2 REAL HARDWARE VALIDATION RESULTS

5.2.1 M0 — WiFi Assessment

M0 was run against real 802.11 beacon traffic using the Alfa AWUS036ACM in genuine monitor mode. In the session reported here, four real campus networks were captured; all four were WPA2-secured, with real, measured RSSI values, and the WSPS framework (Section 4.4.1) scored the strongest of them at Grade D (score 54) — a real, non-synthetic result rather than a constructed example, and a useful one precisely because it is not

a best-case result: it shows the scoring formula correctly penalizing a network that is encrypted but otherwise weakly configured, rather than only ever producing high grades on networks already known to be well configured.

5.2.2 M1 — Network Discovery

M1 was run against a single real, explicitly authorized target on a live network. The target's real MAC address was resolved via ARP even though the device had ICMP blocked, and a real Nmap scan completed against it running as root. The scan's result was an honest negative — no open IoT-relevant ports — because the authorized target available for this session was a general-purpose laptop rather than an IoT device. This negative result is reported here deliberately: a system that only ever reports positive findings during development is weaker evidence than one that is shown, on a real target, to correctly report nothing when there is nothing to find.

5.2.3 M2 — Protocol Fingerprinting

M2 was run against a real, controlled local IoT-service target exposing live HTTP and RTSP behaviour. All four protocol probes (HTTP, MQTT, CoAP, RTSP) ran as real network probes rather than against mocked responses, feeding the additive-confidence fusion algorithm described in Section 4.4.3. The result was `device_type = "IP Camera"` at a fused confidence of 1.0, with structured identity fields (product, version) extracted directly from real response banners — a result traceable, feature by feature, to a named rule in the fusion engine rather than to an opaque model score.

5.2.4 M3 — Vulnerability Intelligence

M3 was validated in both directions that matter for a system whose central discipline is never fabricating a match. Against a synthetic banner with no corresponding entry in the live NVD CPE Dictionary, M3 correctly returned `NO_CPE_DATA` rather than forcing a low-confidence guess. Against a real reference vulnerability, CVE-2017-7577, M3 returned a real `AFFECTED` verdict from a genuine CPE match, with a real CVSS score of 9.8 and a real EPSS score of 29% retrieved live from FIRST.org — both the positive and the negative result were obtained from live NVD, EPSS, and KEV calls, not from cached or mocked data.

5.2.5 M4 — Verification and Gated Exploitation

M4's software path — the authorization gate described in Section 4.6 — was tested adversarially against the real, running dashboard via live HTTP requests, with results reported in Section 5.4. What has not yet been obtained is a positive result against a

genuinely vulnerable physical device: every real M3 result collected so far is an honest non-match or a reference CVE rather than a device acquired specifically for this purpose. This is stated as a known limitation in Section 5.6 rather than implied to be complete.

5.2.6 M5 — Dashboard / API

Every route backing the stages above was verified end-to-end over live HTTP against the running dashboard, in both its real-assessment mode and a dedicated demo mode. The demo mode exists specifically to present pipeline states not yet reached on real hardware — ‘AFFECTED’, ‘VERIFIED_VULNERABLE’, ‘EXPLOIT_SUCCESSFUL’ — for review purposes, without ever presenting simulated data as real: a persistent on-screen banner reads "DEMO MODE — SIMULATED / REPLAY DATA", demo sessions are stored in a structurally separate database file never mixed with real assessment data, every state shown is produced by the real ‘applicability.py’, ‘verification.py’, and ‘exploitation.py’ functions driven by real, previously captured CVE data rather than by a separate mock code path, and simulated exploitation results are labelled "[DEMO — SIMULATED, NOT A REAL EXPLOIT]" in the persisted text itself, not only in the surrounding UI chrome.

5.3 AUTOMATED TEST SUITE SUMMARY

At the point reported in this chapter, the project's automated test suite comprised 381 tests across twenty-eight test files spanning every implemented module (‘tests/m0/’ through ‘tests/m5/’, plus ‘tests/db/’), all passing, run continuously in GitHub Actions on every push as described in Section 4.7. Table 5.2 breaks this count down by module, collected directly from the live test suite rather than carried forward from an earlier status report.

Table 5.2: Automated test count by module (live pytest --collect-only)

Module	Test count
db — SQLite schema and query layer	43
M0 — WiFi assessment (beacon capture, WSPS, OUI-CVE)	39
M1 — network discovery	30
M2 — protocol fingerprinting	41
M3 — CVE correlation, applicability, verification	168
M4 — authorization gate	18
M5 — dashboard / API routes	42
Total	381

M3's share of the suite (168 of 381 tests, 44%) reflects where the project's engineering effort has been concentrated: it is the module responsible for never letting a CVE keyword

match be reported as a confirmed vulnerability (Section 4.4.4), and the applicability and verification logic that enforces this is tested far more exhaustively than any other single module, consistent with it being the load-bearing component behind both Novelty Claim 4 (Section 3.4.4) and the adversarial gate results in Section 5.4. This count is reported as a scale indicator rather than a correctness proof on its own: a mocked unit test confirms internal consistency, not that the mocked behaviour matches a real external system, which is precisely why Section 5.2's real-hardware results are kept visibly distinct from, rather than substituted for, this count.

5.4 SECURITY GATE — ADVERSARIAL TESTING RESULTS

Because Module 4's authorization gate (Section 4.6) is the one component of PISA that carries direct ethical and legal weight, it was tested adversarially rather than only unit-tested in isolation: each row in Table 5.3 is a real HTTP request issued against the real, running dashboard, not a call directly into the gate function in a test harness.

Table 5.3: Adversarial gate testing results (live HTTP against the running dashboard)

Condition exercised	Result
Exploit attempted on a NOT_APPLICABLE finding	HTTP 403, gate_state = BLOCKED_APPLICABILITY, zero exploit_results rows created
Exploit attempted on an AFFECTED but NOT_VERIFIED finding	HTTP 403, gate_state = BLOCKED_VERIFICATION
Authorization fields missing, even on a fully eligible finding	HTTP 400, request blocked before any network action was taken
Client supplies an arbitrary / attacker-chosen module_path	Value silently ignored; the real, registry-resolved module executes instead, never the client's
A genuine RouterSploit execution timeout occurs	Persisted as its own TIMEOUT status; never collapsed into a generic failure result

The fourth row in Table 5.3 is the most security-significant of the five: it confirms that the gate's module-selection logic is not merely validated client-side but is re-derived server-side from the already-verified CVE, so that a request crafted to specify a different, unvetted module path cannot cause that module to run. This directly substantiates the design claim made in Section 4.6 — that no single module's output is independently sufficient to unlock exploitation — against a live adversarial input rather than only against the code as written.

5.5 DISCUSSION

Returning to the research questions posed in Section 3.2: RQ2 (can passive beacon analysis alone produce a meaningful posture score) is supported by the M0 result in Section 5.2.1, where WSPS correctly distinguished a weakly configured, though encrypted, real network from a hypothetical stronger one, without any active probing — though a single real session is not yet sufficient evidence to confirm Hypothesis H1's stronger claim of expert-level agreement, which remains for the evaluation planned ahead of the final review (Chapter 6). RQ3 (can non-ML, protocol-behavioural signatures identify a device at usable accuracy) is directly supported by the M2 result in Section 5.2.3: a confidence of 1.0 on a real IP camera obtained through deterministic, rule-traceable fusion rather than a learned classifier — consistent with, though not yet a controlled comparison against, the ML accuracy baseline reported in the literature [15]. RQ4 (integration accuracy and completeness) is partially addressed: M3's both-directions result in Section 5.2.4 demonstrates the pipeline can produce a correct real CVE match without ever fabricating one, directly supporting the "no single metric is sufficient on its own" argument from Section 2.5 [17], but the timed comparison against a manual multi-tool workflow implied by Hypothesis H4 has not yet been run and is scoped into the roadmap in Chapter 6.

Table 5.4 summarizes this hypothesis-by-hypothesis status explicitly, against the metrics and targets already defined in Table 3.1, so that "supported," "partially supported," and "not yet evaluated" are read as this report defines them rather than informally.

Table 5.4: Hypothesis validation status at the current review

Hypothesis	Status	Basis
H1 — WSPS grades agree with expert assessment	Not yet formally evaluated	A real WSPS grade (D, score 54) was produced correctly (Section 5.2.1); the multi-network expert-agreement study itself is scoped for Chapter 6
H2 — Fingerprinting accuracy competitive with ML baselines	Partially supported	A real, single-device result (confidence 1.0, Section 5.2.3) is consistent with H2; a multi-device accuracy study is scoped for Chapter 6
H3 — Applicable high-severity CVEs surfaced quickly	Partially supported	M3 returned a correct real match and a correct real non-match (Section 5.2.4); timed completeness at scale is scoped for Chapter 6
H4 — End-to-end time substantially below manual workflow	Not yet evaluated	Requires the timed comparison against a manual multi-tool baseline, scoped for Chapter 6

Measured against the novelty claims in Section 3.4, Claim 4's strongest substantiation to date is the adversarial gate-testing result in Section 5.4: the project's central claim is not merely that PISA chains more stages than prior tools, but that it never lets a later stage overclaim what an earlier one actually established, and Table 5.3's module-substitution test is direct, live evidence of exactly that property holding under an adversarial input, not only under the inputs the system's own developers chose to test with.

Taken together, Sections 5.1 through 5.4 support a specific, bounded claim: that the pipeline proposed in Chapter 4 works correctly on real inputs, in both its positive and negative cases, and that its single most safety-critical control — the exploitation authorization gate — holds under adversarial live testing, not only under the conditions its own developers anticipated. They do not yet support the broader, statistical claims embedded in Hypotheses H1-H4, and Table 5.4 states that distinction explicitly rather than letting the two kinds of evidence blur together.

5.6 KNOWN LIMITATIONS

The following limitations are stated proactively rather than discovered by a reviewer, consistent with the reporting discipline set out in Chapter 3.

- **No physically vulnerable IoT device has been tested end-to-end yet.** Every real M3 result obtained so far (Section 5.2.4) is an honest non-match or a reference CVE rather than a live device acquired specifically for this purpose. A target hardware family with demonstrated real NVD CPE-match potential has been identified and scoped but not yet acquired; acquiring and running the complete positive M1-through-M4 chain against it is the single highest-priority item in the roadmap (Chapter 6).
- **M1's discovery has no built-in target-range restriction.** This is a genuine, acknowledged architecture note treated as an authorization-scope question during development rather than an oversight glossed over in this report: nothing in M1 today prevents a discovery sweep from extending beyond the specific range an operator was authorized to assess, which is addressed procedurally today (the ethics framework in Section 3.6) rather than architecturally.
- **RouterSploit's execution timeout is thread-bound, not process-isolated.** A documented limitation of running RouterSploit modules inside CPython threads rather than separate processes; acceptable for a supervised, operator-present assessment of the kind M4 is designed for, but a constraint worth stating rather than assuming away.

- **Cloud reporting remains an intentional stub.** Unchanged since the project's initial scope (Chapter 1); ``pisa/aws/`` exists as a package with no Lambda, DynamoDB, or S3 wiring implemented, and is treated entirely as future work in Chapter 6 rather than partially claimed here.

5.7 ILLUSTRATIVE END-TO-END SESSION WALKTHROUGH

To make the results reported in Sections 5.1 through 5.4 concrete as a single pipeline rather than six isolated results, this section traces one assessment session end to end, using only the real findings already reported above — no new claim is introduced here, only their sequence. An operator starts a scan on the monitor-mode interface; M0 captures real beacon traffic and grades the strongest of four real captured networks at WSPS Grade D (score 54, Section 5.2.1). The operator supplies that network's password and triggers M1's join-and-discover action; an ARP sweep and a pooled Nmap scan enumerate the live hosts on the joined subnet (Section 5.2.2). For a device exposing HTTP and RTSP services, the operator triggers M2's fingerprint action; the four protocol probes return ``device_type = "IP Camera"``` at a fused confidence of 1.0 (Section 5.2.3). The operator then triggers M3's CVE-lookup action; for a device matching the reference finding used throughout this report, M3 returns CVE-2017-7577 with an ``AFFECTED`` applicability verdict, CVSS 9.8, and a live EPSS score of 29% (Section 5.2.4), from which Section 4.4.4's worked example computes an exploit score of roughly 51 (or 71, if KEV-listed) out of 100. Only at this point does Module 4's authorization gate become relevant: Table 5.3 shows that this finding would be blocked from exploitation until M3 additionally returns ``VERIFIED_VULNERABLE`` from a live verification test, and that even then, exploitation requires a named operator, an explicit confirmation, and a five-second delay before it can proceed at all. Every stage above is recorded in the dashboard and persisted to the local database (Section 4.5) independently of whether the operator continues to the next one, so the session's evidence trail survives even if, as in the real validation run reported in Section 5.2.5, the physical exploitation step itself has not yet been reached.

5.8 THREATS TO VALIDITY

Two threats to the validity of the results in this chapter are stated explicitly rather than left implicit. First, **single-session evidence**: several of the real-hardware results reported in Section 5.2 (the WSPS grade on four campus networks, the fingerprinting confidence on one IP camera) are drawn from individual assessment sessions rather than a repeated, multi-

session sample; they demonstrate that the pipeline produces correct results on real inputs, which is a necessary condition for the hypotheses in Section 3.2, but they do not yet constitute the statistical evidence — expert-agreement scores, accuracy across many device instances — those hypotheses ultimately require, and which Table 3.1's planned metrics are designed to collect. Second, **target availability**: the negative M1 result and the absence of a positive M4 exploitation result (Sections 5.2.2 and 5.2.5) both trace to the same underlying constraint, that the specific devices authorized for testing during this phase did not include a confirmed-vulnerable IoT target; this is a constraint on what could be validated, not a result suggesting the pipeline itself is incomplete, and is addressed directly by Phase A of the roadmap in Chapter 6. Neither threat is treated as resolved by this report — both are carried forward explicitly into Chapter 6 rather than left as an implicit caveat.

CHAPTER 6

CONCLUSION AND SCOPE FOR FUTURE WORK

6.1 CONCLUSION

This report has described PISA, a portable IoT security assessment platform that chains passive WiFi posture scoring, device discovery, non-machine-learning protocol fingerprinting, tri-metric CVE correlation, and gated exploit verification into a single pipeline on commodity, field-deployable hardware. Chapter 1 established that no existing tool closes the specific gap between network-layer and device-layer IoT security assessment; Chapter 2 grounded that claim in twenty peer-reviewed and pre-print sources and positioned PISA against nine named prior tools and platforms; Chapter 3 formalized the problem into research questions, five specific novelty claims, and a requirements specification; Chapter 4 described the resulting six-module architecture in full; and Chapter 5 reported, honestly and with limitations stated proactively, what has actually been validated against real hardware, real networks, and real external APIs as of this review.

Every module proposed at the start of this phase — M0 through M5 — is implemented, covered by an automated test suite of 381 tests, and has been exercised at least once against real, non-synthetic conditions rather than only against mocked data. The one component explicitly out of scope, cloud reporting, remains an intentional, clearly labelled stub rather than an unfinished claim. What remains before the system can be considered complete is real-world scale — acquiring and testing against a genuinely vulnerable physical device, and deploying the full pipeline onto its target Raspberry Pi form factor — rather than any open architectural question.

The discipline this report has tried to hold itself to throughout — stating what is implemented against what is planned (Chapter 3), what is real against what is simulated-for-demonstration (Chapter 4), and what is validated against what remains to be measured (Chapter 5) — is not incidental to the project's engineering quality; it is argued here to be part of it. A system whose own documentation cannot be trusted to state its limitations accurately is a weaker foundation for a security tool specifically, where an assessor downstream will act on PISA's output as if it were ground truth. The five novelty claims in Chapter 3 are offered in that spirit: as specific, checkable commitments against named prior work, not as a general assertion of originality.

6.2 SUMMARY OF CONTRIBUTIONS

- A WiFi Security Posture Scoring (WSPS) framework that grades a network from passive beacon analysis alone, validated against four real campus networks.
- A non-machine-learning, cross-protocol device fingerprinting engine (MQTT, CoAP, HTTP, RTSP) with a probabilistic confidence-fusion rule, validated at confidence 1.0 against a real IP camera target.
- A tri-metric (CVSS + EPSS + CISA KEV) exploit-scoring formula computed on edge hardware, validated in both directions — an honest non-match and a real, correctly scored reference CVE — against live NVD, FIRST.org, and CISA data.
- A deterministic applicability engine that evaluates real NVD configuration logic rather than treating a keyword match as a finding, structurally preventing a later pipeline stage from overclaiming what an earlier one established.
- A software-enforced, adversarially-tested authorization gate for exploit verification, shown in Chapter 5 to correctly block every unauthorized or under-evidenced exploitation attempt tried against the live running system.

6.3 ROADMAP TO THE FINAL REVIEW

The work remaining before the final project review is organized into five phases, summarized in Table 6.1. These follow directly from the limitations stated proactively in Section 5.6 rather than introducing new scope at this stage.

Table 6.1: Roadmap to the final review

Phase	Goal
A	Acquire the identified physical vulnerable IoT target hardware
B	Run the complete, real, positive M1-M4 chain on real hardware — real CPE match, real AFFECTED verdict, real VERIFIED_VULNERABLE result, real authorized exploitation
C	Deploy the full pipeline onto a Raspberry Pi 4 in its target field form factor
D	Expand the verified-exploit registry beyond its current entries, each vetted with the same rigour as the existing ones
E	Final report and full end-to-end demonstration rehearsal

6.4 FUTURE WORK BEYOND THE FINAL REVIEW

Several extensions were deliberately scoped out of v1 to keep the current phase's claims fully verifiable against real evidence, and remain candidates for future work rather than abandoned ideas.

- **Cloud reporting (AWS).** Synchronizing session data to DynamoDB and S3 and generating automated PDF reports via Lambda, as scoped in Chapter 4, remains the most immediately actionable extension, since the local SQLite schema already carries the data such a sync would need.
- **Continuous monitoring mode.** Running M0 indefinitely in the background and alerting on new devices, WSPS score degradation, or newly published CVEs matching an already-fingerprinted device, converting PISA from a point-in-time audit tool into a longitudinal monitor.
- **Behavioral drift detection.** The `behavioral\_drift` table (Table 4.4) is already reserved in the schema; populating it by comparing a device's fingerprint across sessions would flag firmware updates or possible device impersonation.
- **GPS-tagged assessment and a dedicated touchscreen interface,** removing the current dependency on a separate laptop or phone browser for field operation, as scoped in Table 4.1.
- **Formal evaluation against Hypotheses H1-H4** (Section 3.2), measured against the metrics already defined in Table 3.1 — an expert-agreement study for WSPS, a multi-device fingerprinting accuracy study, and a timed comparison against a manual multi-tool workflow — once the physical hardware acquisition in Phase A above is complete.
- **Modbus behavioural fingerprinting,** extending Module 2's protocol coverage to industrial/OT devices (PLCs, SCADA endpoints) by the same deterministic, non-ML probing approach used for MQTT, CoAP, HTTP, and RTSP.
- **A multi-device fleet mode,** where several PISA units report findings to one shared signature database without centralizing raw traffic — extending the single-device assessment reported in this document toward the kind of privacy-preserving, federated architecture reviewed in Section 2.3.

REFERENCES

The following references are listed in alphabetical order of the first author's surname, as required by the project report format guidelines. In-text citations throughout this report refer to these numbers in square brackets.

- [1] T. Amsuess, C. Amsuss, T. Fossati, and M. Tiloca, "Securing Name Resolution in IoT: DNS over CoAP," *ACM Transactions on Internet of Things*, vol. 1, no. 2, Article 9, 2023, doi: 10.1145/3609423.
- [2] M. Anand, M. Joshi, and N. Chirculescu, "A New Method for Vulnerability and Risk Assessment of IoT," *Computer Networks* (Elsevier), vol. 236, Article 109788, 2023, doi: 10.1016/j.comnet.2023.109788.
- [3] A. Churcher, R. Ullah, J. Ahmad, S. ur Rehman, et al., "Survey on Enterprise IoT Systems (E-IoT): A Security Perspective," *IEEE Access*, vol. 9, pp. 62289-62317, 2021, doi: 10.1109/ACCESS.2021.3073730.
- [4] X. Dong, J. Zheng, Z. Li, and Y. Chen, "Toward IoT Device Fingerprinting from Proprietary Protocol Traffic," *Computer Networks* (Elsevier), vol. 224, Article 109612, 2023, doi: 10.1016/j.comnet.2023.109612.
- [5] E. Gvozdenovic, J. Bridwell, J. Weston, and D. Raje, "Systematically Analyzing Vulnerabilities in Wi-Fi Connection Establishment," in *Proc. IEEE Conf. on Communications and Network Security (CNS)*, 2022.
- [6] S. Habibi Lashkari, A. Zino, and A. Tavallaei, "Device Identification Using Optimized Digital Footprints," *arXiv preprint arXiv:2212.04354*, 2022.
- [7] M. Hasan, M. Islam, M. Zarif, and M. Hashem, "Preventing MQTT Vulnerabilities Using IoT-Enabled Intrusion Detection," *Sensors* (MDPI), vol. 22, no. 2, Article 567, 2022, doi: 10.3390/s22020567.
- [8] G. Heiding, B. Lundell, and U. Lindqvist, "SAFER: IoT Device Risk Assessment Framework in a Multinational Organization," *arXiv preprint arXiv:2007.14724*, 2020.
- [9] J. Jacobs, S. Romanosky, B. Edwards, M. Roytman, and I. Adjerid, "Exploit Prediction Scoring System (EPSS)," *ACM Digital Threats: Research and Practice*, vol. 2, no. 3, Article 16, 2021, doi: 10.1145/3436242.

- [10] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DIoT: A Self-Learning System for Detecting Compromised IoT Devices," *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 3851-3863, 2023, doi: 10.1109/JIOT.2022.3199104.
- [11] M. Palmieri, M. Cebe, C. Ozmen-Ertekin, et al., "Experimental Evaluation of MQTT Authentication and Authorization in IoT," in *Proc. ACM WiNTECH Workshop (co-located with ACM MobiCom)*, 2021, doi: 10.1145/3477086.3480838.
- [12] P. Radoglou-Grammatikis, P. Sarigiannidis, G. Efstathopoulos, P. Karypidis, and A. Sarigiannidis, "Analysis of Consumer IoT Device Vulnerability Quantification Frameworks," *Electronics* (MDPI), vol. 12, no. 5, Article 1176, 2023, doi: 10.3390/electronics12051176.
- [13] J. Ren, D. J. Dubois, D. Choffnes, A. Mandalari, R. Kolcun, and H. Haddadi, "Analyzing Consumer IoT Traffic: Security and Privacy Perspectives," *arXiv preprint arXiv:2403.16149*, 2024.
- [14] D. Schepers, A. Ranganathan, and M. Vanhoef, "Framing Frames: Bypassing Wi-Fi Encryption via Transmit Queue Manipulation," in *Proc. 32nd USENIX Security Symposium*, 2023.
- [15] A. Sivanathan, H. H. Gharakheili, and V. Sivaraman, "IoT Behavioral Monitoring via Network Traffic Analysis," *arXiv preprint arXiv:2001.10632*, 2020.
- [16] M. Soni and A. Singh, "Vulnerability Assessment of MQTT Protocol in Internet of Things (IoT)," in *Proc. IEEE Int. Conf. on Computing, Communication and Automation (ICCCA)*, 2021, doi: 10.1109/ICCCA52192.2021.9478156.
- [17] A. Subramaniam, S. Krishnan, and R. Vitale, "Efficacy of EPSS in High-Severity CVEs Found in CISA KEV," *arXiv preprint arXiv:2411.02618*, 2024.
- [18] M. Vanhoef, "Fragment and Forge: Breaking Wi-Fi Through Frame Aggregation and Fragmentation," in *Proc. 30th USENIX Security Symposium*, 2021.
- [19] M. Vanhoef and E. Ronen, "Dragonblood: Analyzing the Dragonfly Handshake of WPA3 and EAP-pwd," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 517-533, doi: 10.1109/SP40000.2020.00071.

- [20] J. Varga, A. Kovacs, and P. Szabo, "IoT Device Security Audit Tools: Comprehensive Analysis and Layered Architecture," *International Journal of Information Security* (Springer), 2024, doi: 10.1007/s10207-024-00930-z.

APPENDICES

APPENDIX A — PLAGIARISM REPORT

A similarity/plagiarism report for this project report, generated through the college's approved plagiarism-detection tool, is to be inserted at this appendix prior to final submission, in accordance with the School's plagiarism policy stated in the project report preparation guidelines.

APPENDIX B — REPOSITORY AND DOCUMENTATION INDEX

The complete source code, automated test suite, and underlying project documentation this report is based on are maintained in the project's version-controlled repository. Table references, figures, and all status claims made in Chapters 4 and 5 trace to the following source documents within it:

- ``README.md`` — current module implementation status and operating instructions
- ``docs/PISA_SRS.md`` — formal Software Requirements Specification (IEEE 830-1998)
- ``docs/FYP_PROJECT_DOCUMENTATION.md`` — full research motivation, novelty claims, and system design
- ``docs/PISA_Review1_Report.md`` — real-hardware validation results reported in Chapter 5
- ``docs/papers/PAPERS_INDEX.md`` and ``docs/papers/LITERATURE_SURVEY_NOTES.md`` — the twenty-paper literature base reviewed in Chapter 2
- ``docs/diagrams/`` — source architecture and module diagrams reproduced as Figures 4.1 and 4.2
- ``tests/`` — the 381-test automated suite summarized in Section 5.3